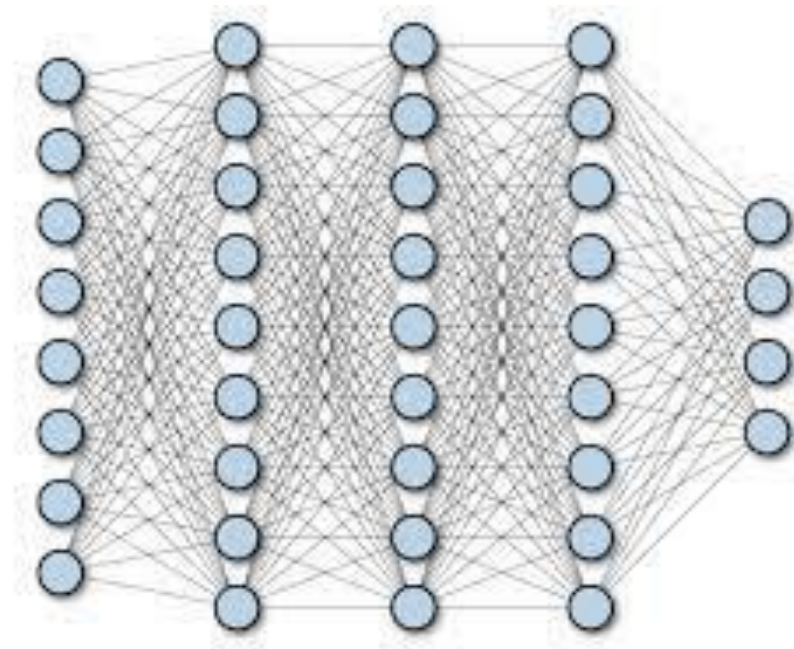
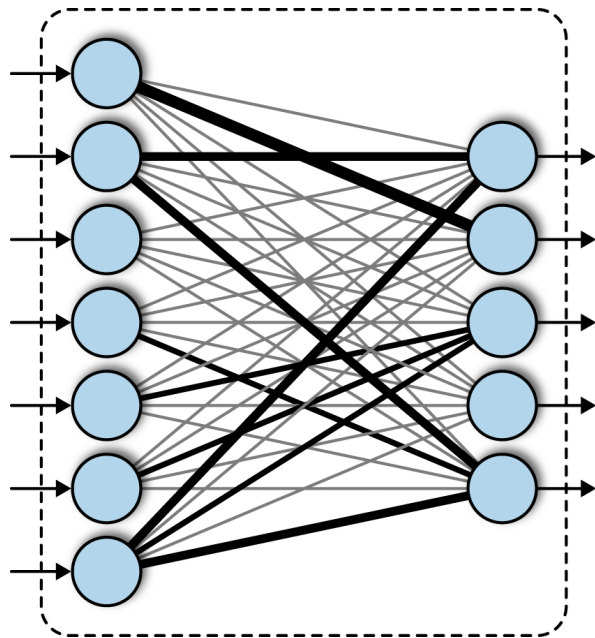


Chapter 2

Fully Connected Neural Networks

(Each neuron is connected to all neurons of the next layer)



1.1.1 Fully Connected layers

Ideal to make partitions (classification problems), maps (Regression Problems) and encoded form of data from a set of distinct inputs.

- One Layer Neural Networks (Perceptron – Adaline - Madaline)
- MLP With one Hidden Layer
- MLP With Two Hidden Layers
- Multilayer Neural Networks
- Auto Encoders
- Restricted Boltzmann Machine
- Deep Belief Neural Networks

Classification problem

- The action or process of classifying something based on its features

$$\{(x_q, t_q)\}_{q=1}^Q \quad x_q: \text{input features} \rightarrow t_q = \text{output labels (class1, class2..class N)}$$

Assumptions:

1. Input Features are represented as a numerical vector with the same length.

$$x_q = [x_{q1}, \dots, x_{qn}]$$

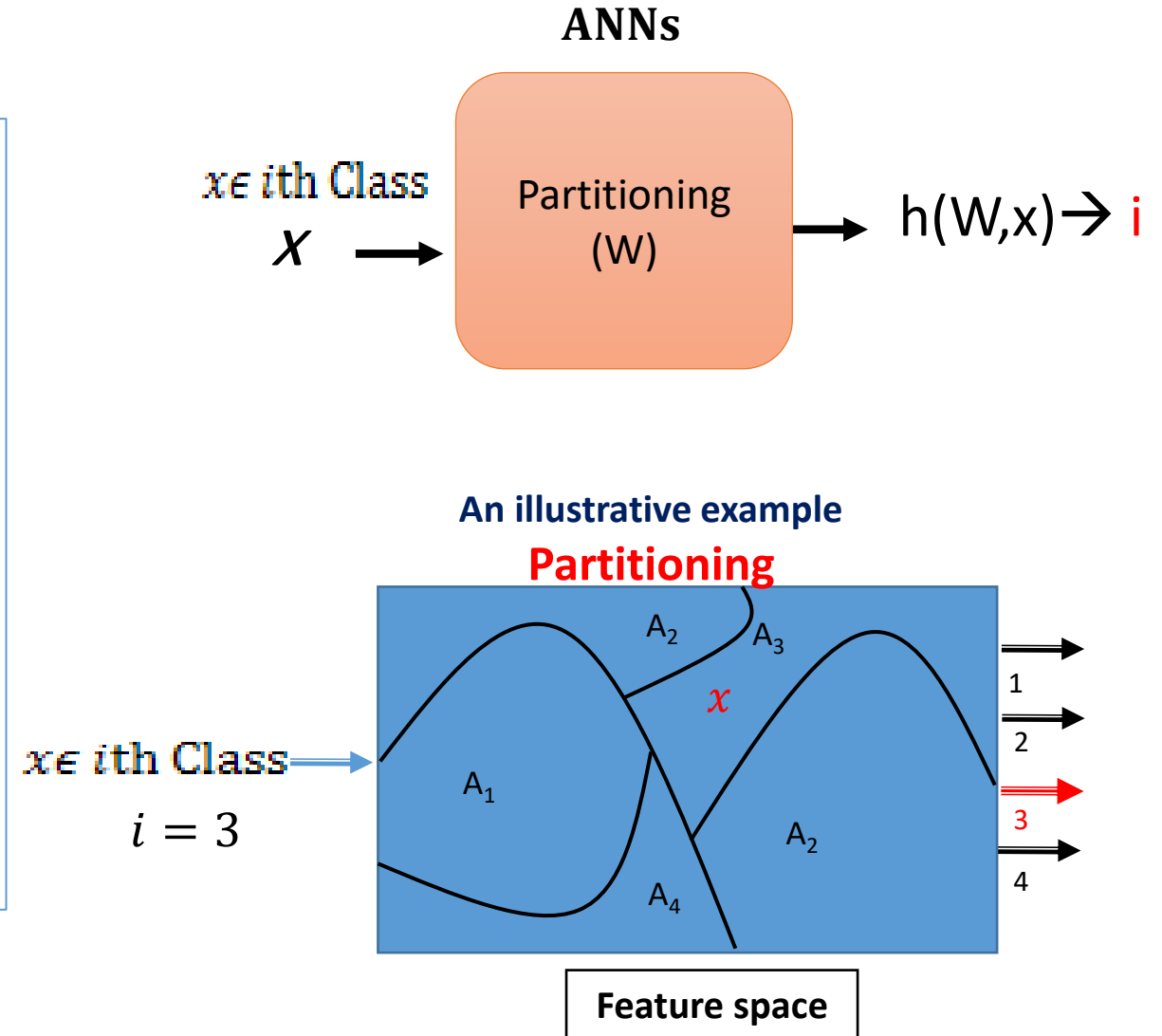
2. The input features are distinct (there is no temporal or spatial correlation).
3. The input features are chosen as relevant features for prediction of the output labels.
4. The labels of classes are represented as one hot, binary or scalar format.

$$N=3 \text{ class, } t_q \in \{[1 \ 0 \ 0], [0 \ 1 \ 0], [0 \ 0 \ 1]\}, t_q \in \{[0 \ 0], [0 \ 1], [1 \ 0]\}, t_q \in \{1, 2, 3\},$$

5. It is assumed that input feature points of each class are near together and make one or more than one cluster (high Separation index).

Classification/Partitioning

- It is desired to learn a ANN (supervised learning) which can partition the input space into regions where data points with the same labels are distributed.
- By partitioning the feature space, the target labels is predicted at output of the network.



Regression problem

- In statistical **modeling**, **regression analysis** is a set of statistical processes for estimating the relationships among variables.

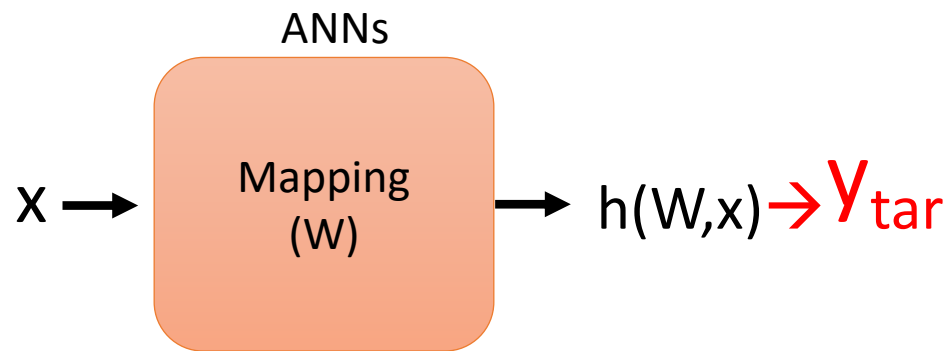
$$\{(x_q, y_q)\}_{q=1}^Q \quad x_q: \text{input variables} \rightarrow y_q = \text{output variables (targets)}$$

Assumptions

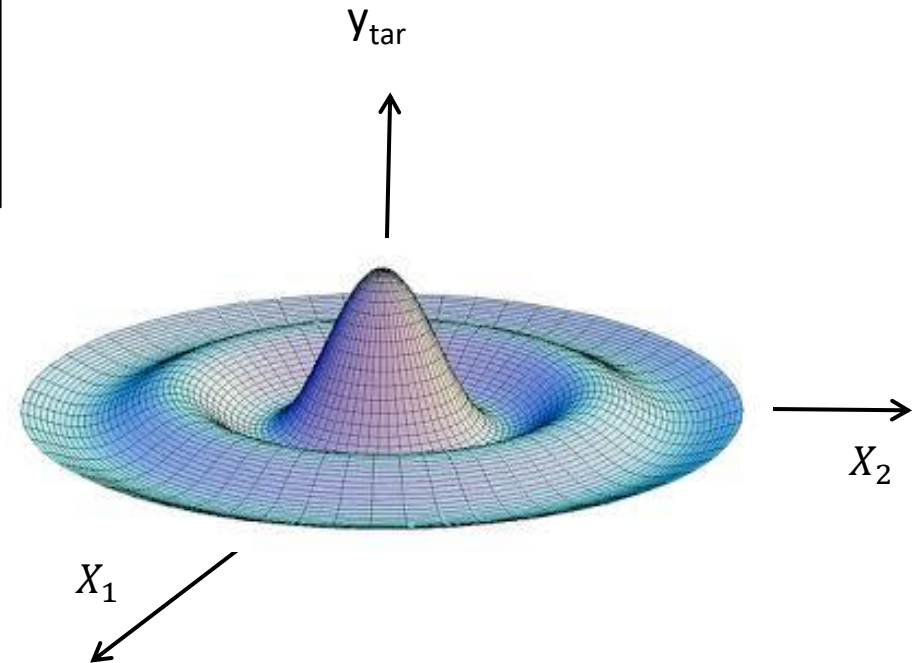
1. Input variables (or input features) can be represented as a numerical vector with the same length.
2. The given input variables are distinct.
3. The output variables are real numbers.
4. They are chosen to be relevant for prediction of the output variables.
5. It is assumed that the input variables provide smoothness for the output variables. Nearness of input variables provide nearness of output variables.

Regression/Mapping

- It is desired to learn a ANN (supervised learning) which can map the input space to target space.
- By mapping the input variables, the output targets are predicted at output of the network.



An illustrative example



Classification versus Regression

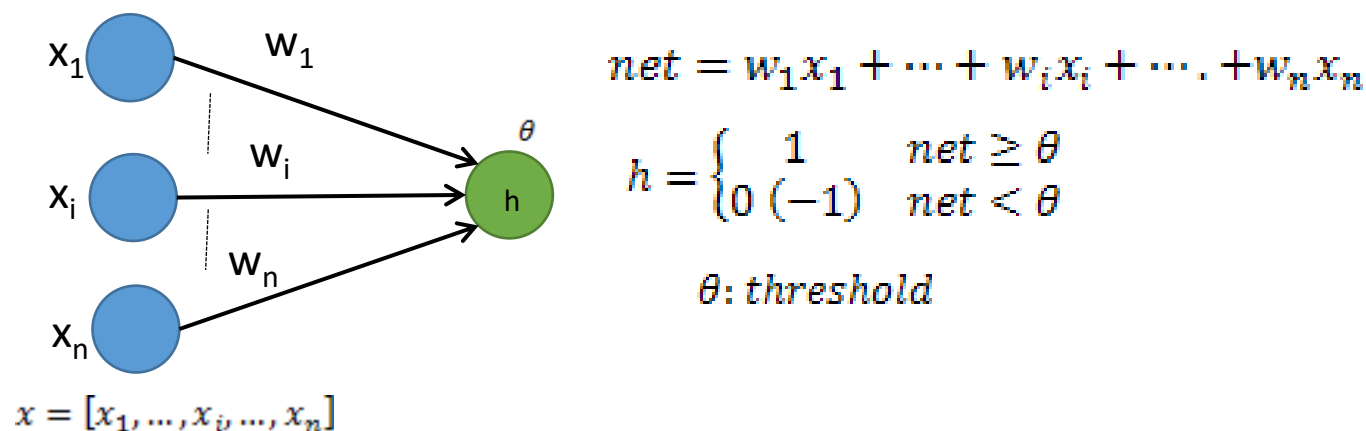
- The **labels** of classes in a classification problem are **quantized** (Discrete in amplitude, (one hot, binary, decimal, etc.).
- The **outputs variables** in a regression problem generally are presented **analogue** (Continues in amplitude).
- A classification problem can be defined as a regression problem whose outputs are quantized.
- A regression problem can be considered a classification problem with an infinite number of classes.
- A regression problem can be approximated as a classification problem by quantizing its outputs.

Chapter 2 – part 1

One-Layer Neural Networks

One-layer NNs for classification

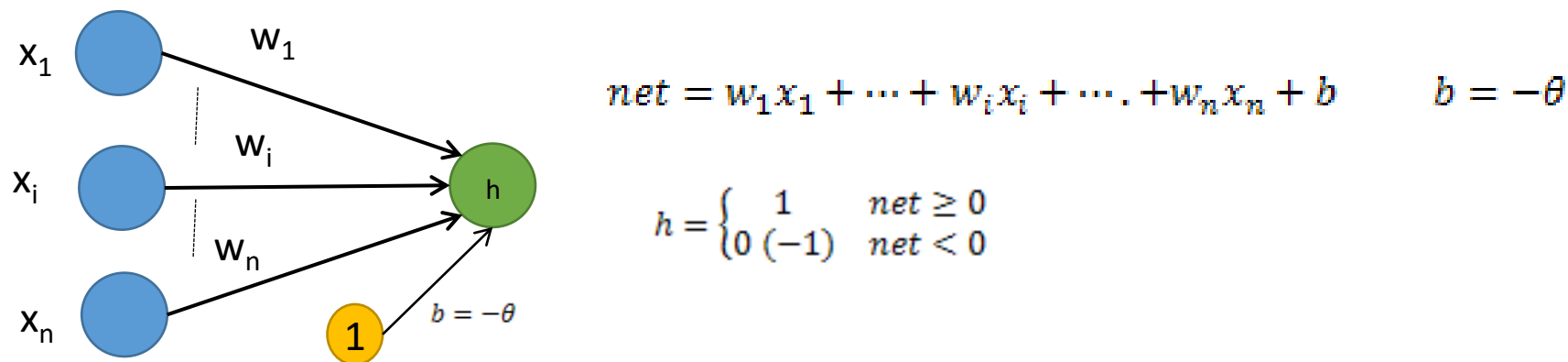
A simple one-layer network with a M&P neuron



Remark

Because of simple structure of one-layer NNs, Such networks only include partitioning layers .

One can replace the threshold of the M&P neuron with “0” by adding as an extra input (bias term) as follows:

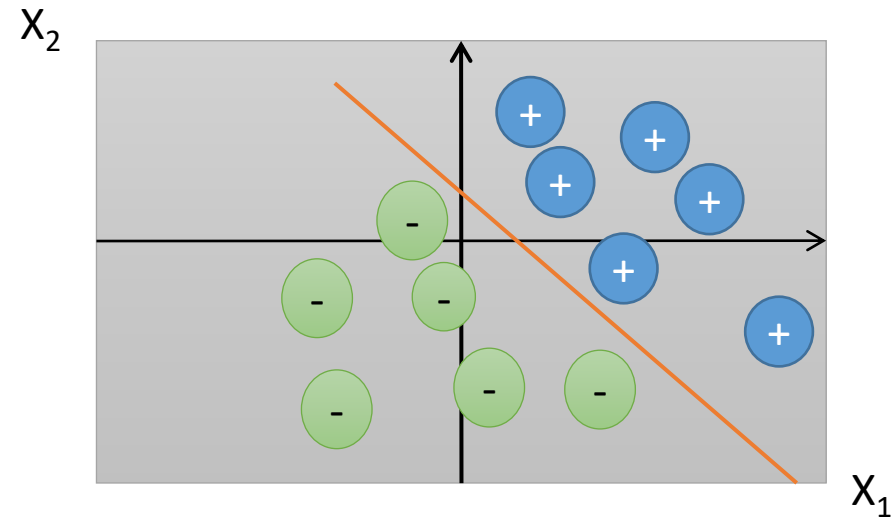
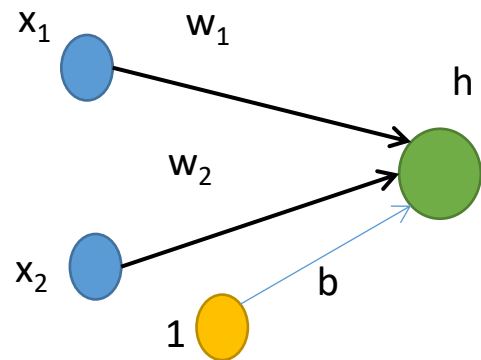


Potential of one-layer NNs in classification

Each M&P neuron can partition the feature space into two regions through using an (n-1) dimensional hyper-plane in the input space.

Example: (n=2) $net = w_1x_1 + w_2x_2 + b$ $b < 0, w_1 > 0, w_2 > 0$

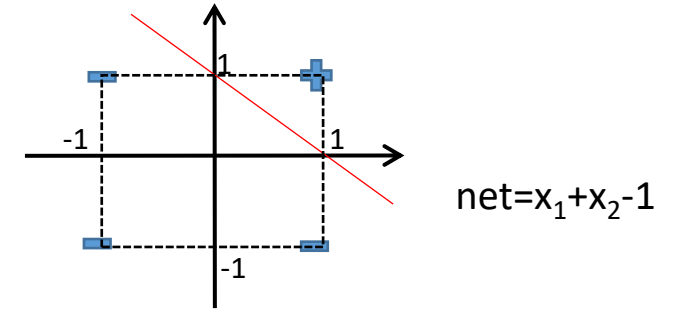
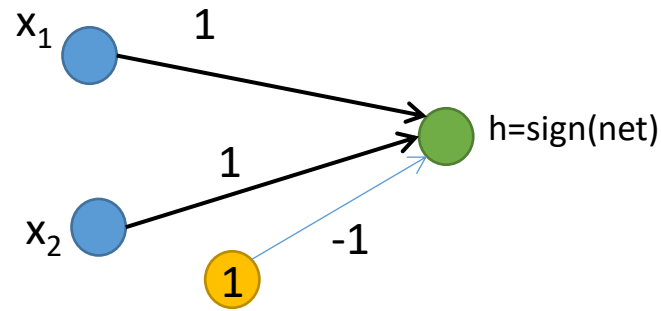
$$h = \begin{cases} 1 & net \geq 0 \quad (+) \\ 0 \quad (-1) & net < 0 \quad (-) \end{cases}$$



More simple Examples (feature points are separated by a line)

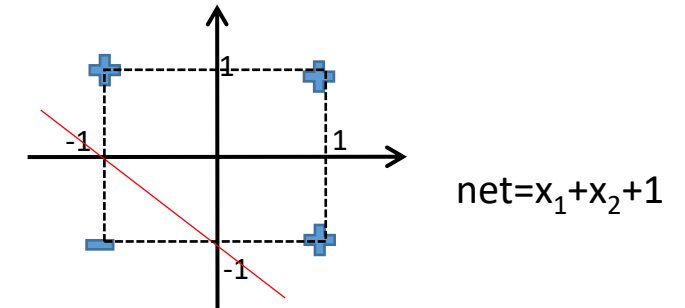
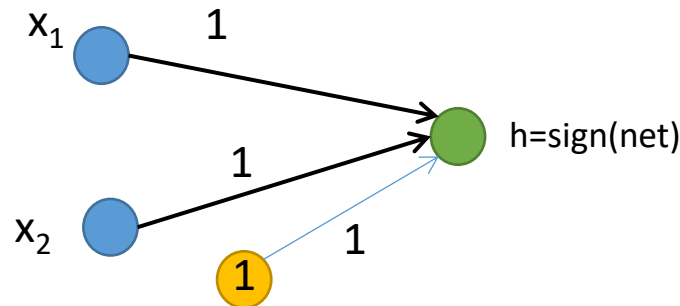
AND Logic

x1	x2	y
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1



OR logic

x1	x2	y
1	1	1
1	-1	1
-1	1	1
-1	-1	-1



1. The separating line between feature points of two classes is determined in a learning process.
2. It is desired that the determined separating line makes enough big margins by boundary feature points of both classes.

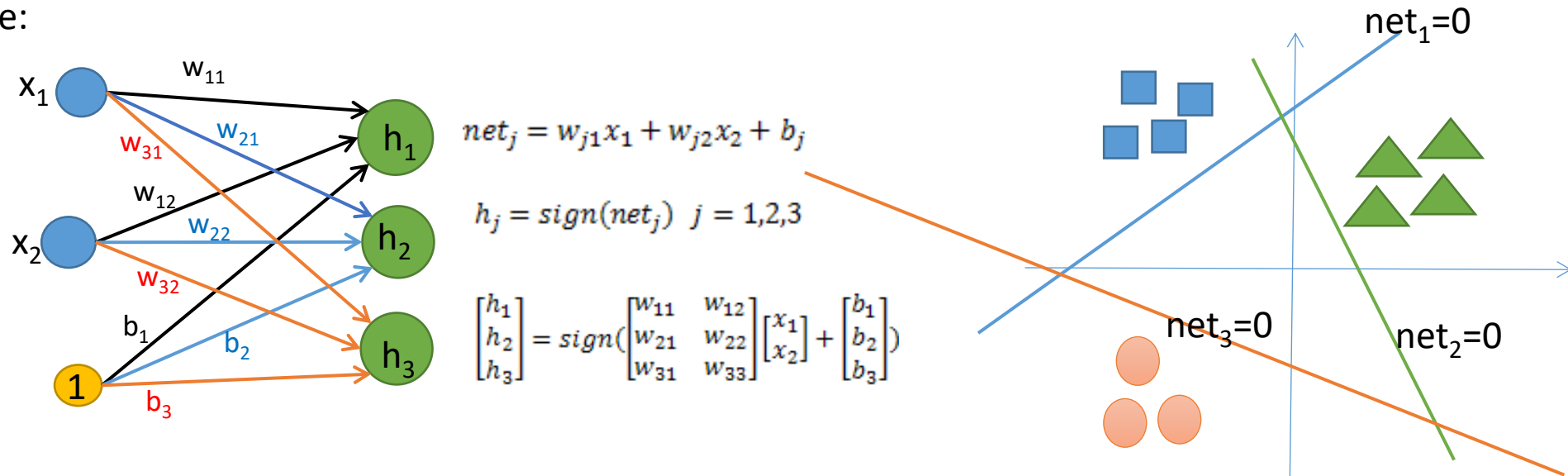
Using one-layer NNs to classify more than two classes

For classifying two different classes, which are linearly separable, one M&P neuron is enough.

Assuming m ($m > 2$) classes can be separated linearly through a simple one-layer NN, we want to classify each class with other ones by an independent output (one-hot outputs):

- 1- Exactly, m M&P neurons are required for classification purpose.
- 2- Each M&P neuron will separate a certain class from other ones by forming a hyper-plane in feature space.
- 3- By using such hyper planes, a convex hyper-polygon is appeared .

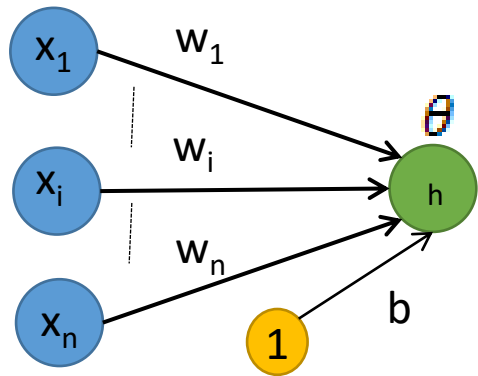
Example:



Chapter 2 – part 1

Linear Perceptron Network

Linear Perceptron Network (Rosenblatt, 1958, 59, 62)

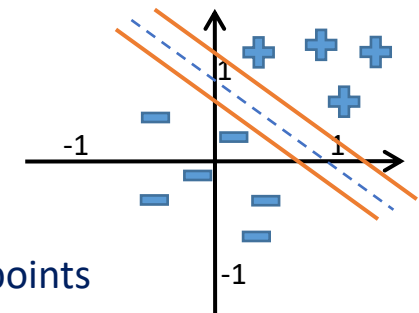


$$\text{net} = w_1x_1 + \dots + w_ix_i + \dots + w_nx_n + b$$

$$(\#) \quad h = \begin{cases} 1 & \text{if } \text{net} > \theta & \text{Active} \\ 0 & \text{if } |\text{net}| < \theta & \text{non - descision} \\ -1 & \text{if } \text{net} < -\theta & \text{Passive} \end{cases}$$

θ : non – negative threshold

Example $\text{net} = x_1 + x_2 - 1$
 $\theta = 0.2$



- In linear perceptron network, two parallel lines (with a distance depended to θ) separate feature points of two classes.
- Bigger threshold (θ) makes larger distance between lines.
- The considered gap 2θ between lines forces a conservative margin in separation of two classes

Learning rules for Perceptron Network

1- Updating rule for the weights:

$$(*) \quad w_i(new) = w_i(old) + \alpha x_i t \quad \alpha : \text{learning rate}$$
$$0 < \alpha \leq 1$$

where x_i denotes i th input and t denotes the target value for output

2-Updating rule for the bias:

$$(**) \quad b(new) = b(old) + \alpha \cdot t$$

- One can show that the above updating rules change the weights and the bias term based on negative-gradient of squared error (loss function).

$\text{sign}(w_i^+ - w_i^-) = -\text{sign}(\text{gradient of squared error with respect to } w)$

$\text{sign}(b^+ - b^-) = -\text{sign}(\text{gradient of squared error with respect to } b)$

Learning Algorithm for Perceptron Network

Step 0- initialize the weights, the bias term, the learning rate and the threshold

$$w_i = 0, \quad b = 0, \quad \theta = 0 \quad \text{and} \quad \alpha = 1$$

Step 1- for each (input , target) training set, $\{(s,t)\}$, do steps 2, 3 and 4.

Step 2- Assign $s_i \rightarrow x_i$ for $i = 1, 2, \dots, n$

Step 3- Compute the output vector, h , according to equation (#)

Step 4- if the error, $(h-t)$, is not zero update the weights and the bias term according to equations (*),(**)

Step 5- if the computed error terms for all training pairs (s, t) are zero go to “End” else go to Step1.

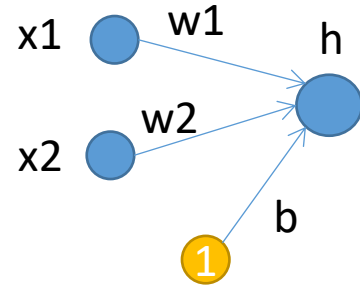
Step 6- End.

Some examples

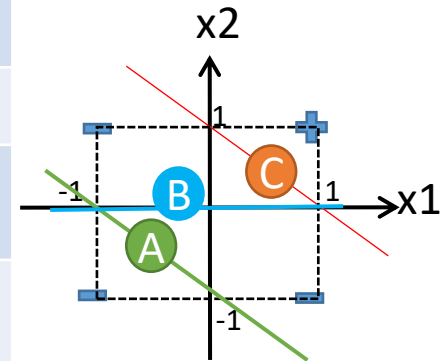
1- Classification of “And logic ” (with bipolar inputs and output) by Linear Perceptron Network

n=2

sample	s		1	t
	x1	x2		
1	1	1	1	1
2	1	-1	1	-1
3	-1	1	1	-1
4	-1	-1	1	-1



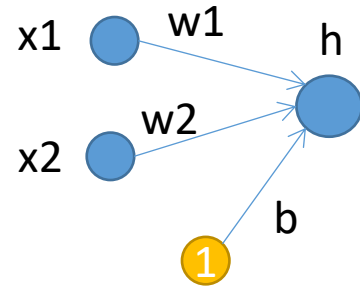
Sample	w1	w2	b	h	error	Line
1	0	0	0	0	-1	
2	1	1	1	1	2	A
3	0	2	0	1	2	B
4	1	1	-1	-1	0	C
1	1	1	-1	-1	0	C
2	1	1	-1	1	0	C
3	1	1	-1	-1	0	C



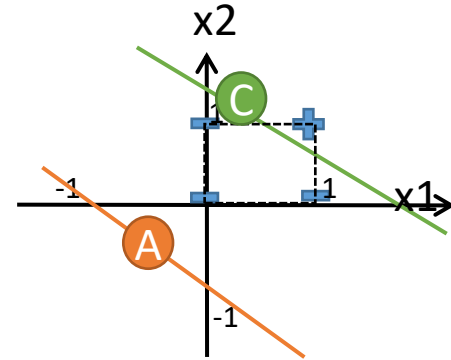
2- Classification of “And logic” (with binary inputs and bipolar output) by Linear Perceptron Network

n=2

sample	s		1	t
	x1	x2		
1	1	1	1	1
2	1	0	1	-1
3	0	1	1	-1
4	0	0	1	-1



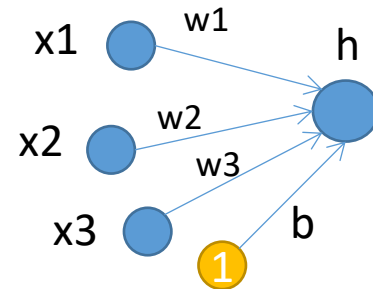
Sample	w1	w2	b	h	error	Line
1	0	0	0	0	-1	
2	1	1	1	1	2	A
.....(after 10 epoch).....						
1	2	3	-4	1	0	C
2	2	3	-4	-1	0	C
3	2	3	-4	-1	0	C
4	2	3	-4	-1	0	C



3- Parity bit

n=3

sample	s			1	t
	x1	x2	x3		
1	1	1	1	1	1
2	1	1	0	1	-1
3	1	0	1	1	-1
4	0	1	1	1	-1



If all bits are “1” then $t=1$ else $t=-1$

Sample 1	w1 0	w2 0	w3 0	b 0	error -1
2	1	1	1	1	2
----- after 26 epochs-----					
1	2	3	4	-8	0
2	2	3	4	-8	0
3	2	3	4	-8	0
4	2	3	4	-8	0

4- Classifying letters A,B,C,D,E,J and K

- Each Letter has three different fonts and totally 21 patterns are defined.
it is desired that each letter with its three patterns being classified.

Each letter is shown through a 9*7 dimensional dot matrix.

#	#	#	#	#	#	0
#	0	0	0	0	0	#
#	0	0	0	0	0	#
#	0	0	0	0	0	#
#	#	#	#	#	#	#
#	0	0	0	0	0	#
#	0	0	0	0	0	#
#	0	0	0	0	0	#
#	#	#	#	#	#	0

is represented by 1

0 is represented by -1

9 rows (each one includes 7 pixels) make a vector with 63 elements

$S_B = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1, \dots, 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1]$

$n=63$

Input from
Font 1

ח

A 10x10 grid of dots. The dots are arranged in a regular pattern. Some dots are replaced by hash symbols (#) to form a pattern. The pattern consists of several vertical and diagonal lines of hash symbols. Specifically, the hash symbols are located at the following (row, column) positions (assuming the top-left dot is (0,0) and the bottom-right dot is (9,9)):

- (0,2), (0,3), (0,6), (0,7)
- (1,1), (1,2), (1,3), (1,6), (1,7)
- (2,1), (2,2), (2,3), (2,6), (2,7)
- (3,1), (3,2), (3,3), (3,6), (3,7)
- (4,1), (4,2), (4,3), (4,6), (4,7)
- (5,1), (5,2), (5,3), (5,6), (5,7)
- (6,1), (6,2), (6,3), (6,6), (6,7)
- (7,1), (7,2), (7,3), (7,6), (7,7)
- (8,1), (8,2), (8,3), (8,6), (8,7)
- (9,1), (9,2), (9,3), (9,6), (9,7)

A.

E

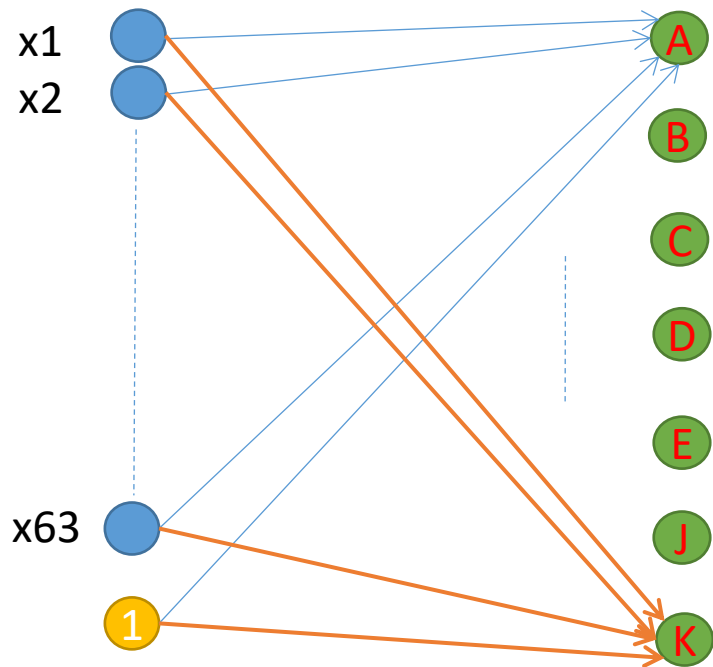
A 6x6 grid of dots forming a stylized letter 'A'. The grid is composed of 36 dots arranged in 6 rows and 6 columns. The dots are positioned at the intersections of the grid lines, forming the shape of the letter 'A'.

. B

.....J.

. . C

.....K



After training phase, each output neuron separates patterns of a certain class with patterns of other classes.

1. it is concluded that the patterns of each class and other 6 classes can be separated, linearly.
2. 7 determined hyper planes form a convex hyper-polygon in input space.

Suppose that all 21 patterns become disturbed with noise: some of pixels #(0) are randomly replaced with 0(#) or become off (up to 30%).

The former trained perceptron Network is applied to classify such disturbed letters. The following table presents the results:

Font1	Disturbed	A	B	C	D	E	J	K
	Output ANN	A	B	C,E,K	D	E	J	K
Font2	Disturbed	A	B	C	D	E	J	K
	Determined	K	B	C	D	E	J	K
Font3	Disturbed	A	B	C	D	E	J	K
	Output ANN	A	B	C	D	E , K	J	K

As it is understood the disturbed patterns of different classes can not be separated linearly.

Perceptron Learning Rule Convergence Theorem

Problem

The perceptron learning rule is as follows:

Given a finite set of P input training vectors

$x(p)$ $p=1,\dots,P$

Each with an associated target value

$t(p)$, $p=1,\dots,P$

Which is either +1 or -1, and an activation function

$h=f(\text{net})$, where

$$h = \begin{cases} 1 & \text{if } \text{net} > \theta & \text{Active} \\ 0 & \text{if } |\text{net}| < \theta & \text{non-decision} \\ -1 & \text{if } \text{net} < -\theta & \text{Passive} \end{cases}$$

The weights are updated as follows

If $(h-t)$ is not zero then

$$w(\text{new}) = w(\text{old}) + tx$$

Else

No change in the weights

Theorem:

If there is a weight vector W^* such that $f(x(p) \cdot W^*) = t(p)$ for all P . then for any starting vector w , the perceptron learning rule will converge to a weight vector (not necessarily unique and not necessarily w^*) that gives the correct response for all training patterns, and it will do so in a finite number of steps.

Proof is given in the book

An upper bound for number of iterations in convergence is resulted as follows:

$$k \leq \frac{M \|w^*\|^2}{m^2}.$$

$M = \max \{\|x\|^2 \text{ for all } x \text{ in the training set}\}$

$$m = \min\{x \cdot w^*\}$$

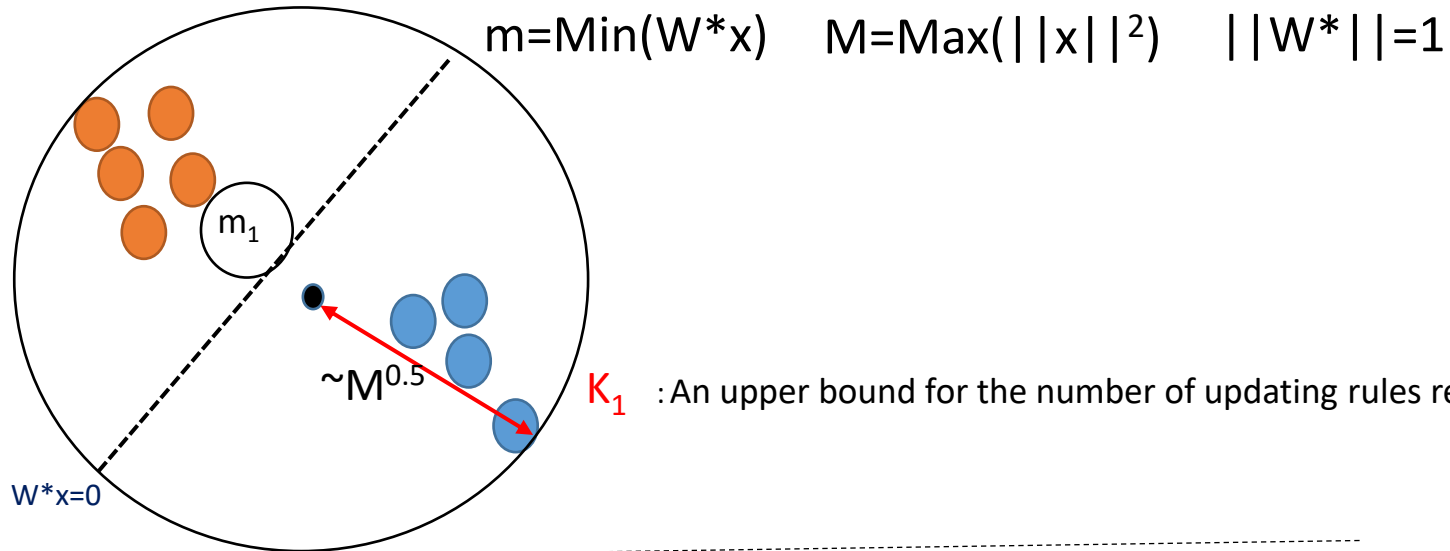
Considering some assumptions it can be concluded that:

1- M addresses a variant of within distance among patterns of each class(WD).

2- m addresses a variant between distance among patterns of two classes(BD)

if in an distribution, all patterns of a class make small WD and patterns of two classes make big BD, the upper bound will be decreased and the convergence may be occurred through less iterations.

Example

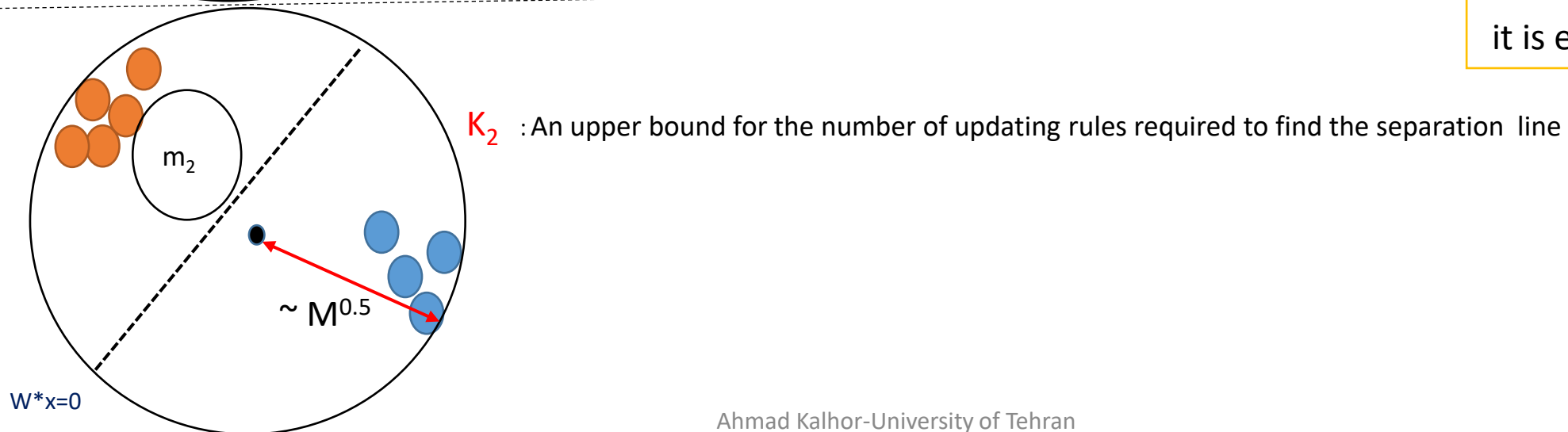


$$k \leq \frac{M \|w^*\|^2}{m^2}.$$

K_1 : An upper bound for the number of updating rules required to find the separation line

Since $m_1 < m_2$

it is expected that $K_1 \gg k_2$

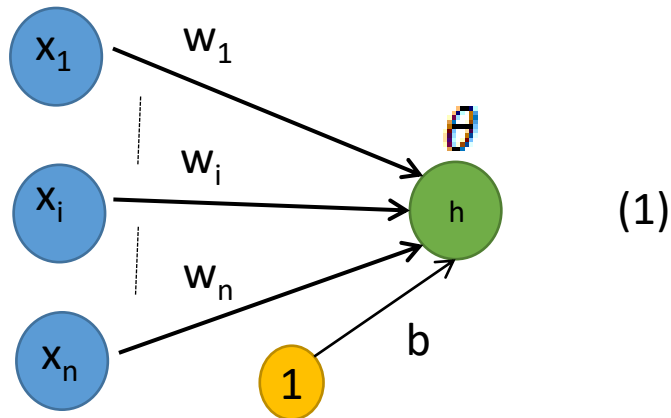


Chapter 2 – part 1

Adaptive Linear Unit (AdaLine)

Adaptive Linear Unit (AdaLine) (widrow & Hoff 1960)

AdaLine is a type of one-layer network



$$\text{net} = \sum_{i=1}^n w_i x_i + b$$

$$h = f(\text{net}) = \begin{cases} +1 & \text{net} \geq 0 \\ -1 & \text{net} < 0 \end{cases}$$

The architecture of AdaLine and Perceptron networks are similar but they use different learning rules for the weights and the bias.

-Here, the activation function does not have threshold.

Learning rules in Adaline

For each training data set (s,t), the updating rules for weights and bias are defined as follows:

$$(2) \quad \begin{aligned} w_i^+ &= w_i^- + \alpha(t - net)x_i \\ b^+ &= b^- + \alpha(t - net) \end{aligned} \quad \alpha > 0 : \alpha \text{ small enough learning rate}$$

Delta is the name of learning rules utilized in AdaLine

Delta's learning rules are defined based on LMS (Least Mean Squares).

error=t-net

Learning Algorithm for AdaLine

Step 0- initialize the weights, the bias term (small random values) and choose a small value for learning rate.

Step 1- for each (input , target) training set, $\{(s,t)\}$,do steps 2, 3 and 4

Step 2- Assign $s_i \rightarrow x_i$ for $i = 1, 2, \dots, n$

Step 3- Compute sum of weighted inputs “net” according to equation (1)

Step 4- Update the weights and the bias term according to equations (2).

Step 5- if the computed cost function for all training pairs (s,t) becomes less than a threshold go to “End” and else go to Step1.

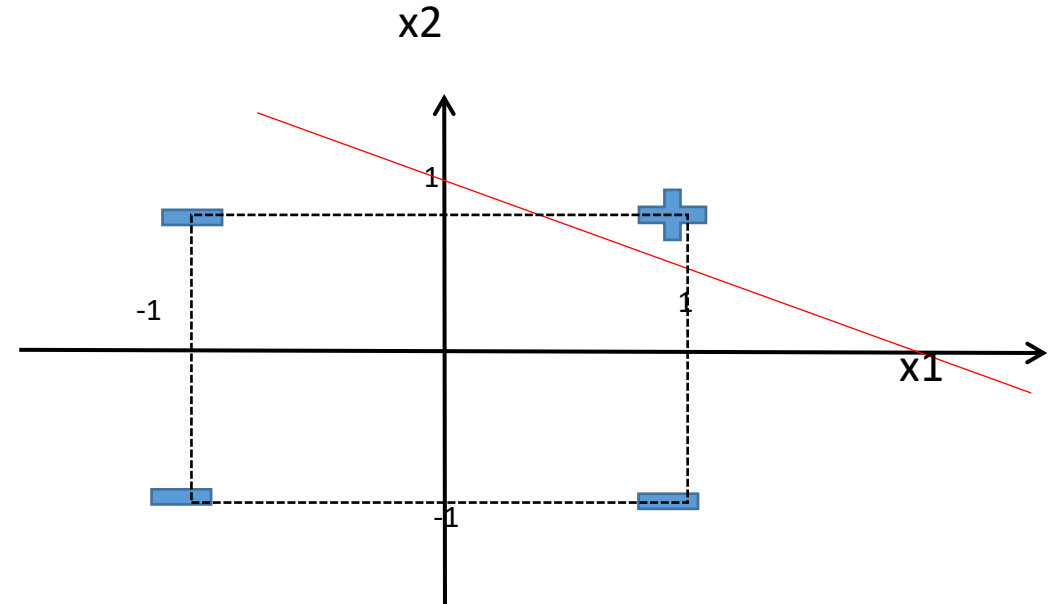
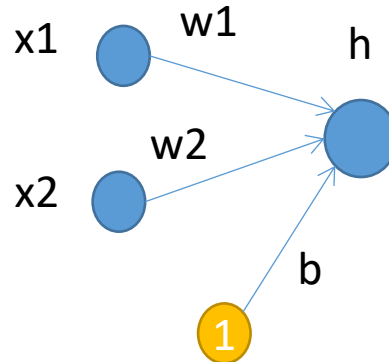
Step 6- End

Example

1- Classification of “And logic ” (with bipolar input and output) by AdaLin.

n=2

sample	s		1	t
	x1	x2		
1	1	1	1	1
2	1	-1	1	-1
3	-1	1	1	-1
4	-1	-1	1	-1



Final optimized line: $\text{net} = \frac{2}{3} x_1 + x_2 - \frac{4}{3}$

Delta works based on the Gradient Descent method

Cost Function: Square Error between the target and net

$$J_p(\mathbf{w}, b) = 0.5(t(p) - \text{net}(x(p), \mathbf{w}, b))^2 \quad \mathbf{w} = [w_1, \dots, w_n] \quad p: \text{sample number of training set}$$

Delta updates the weights and the bias of the AdaLine based on the (first order) Gradient Descent method.

$$\delta w_i = w_i^+ - w_i^- = -\alpha \frac{\partial J_p(\mathbf{w}, b)}{\partial w_i} \Big|_{w^-, b^-} = \alpha(t - \text{net})x_i$$

$$\delta b = b^+ - b^- = -\alpha \frac{\partial J_p(\mathbf{w}, b)}{\partial b} \Big|_{w^-, b^-} = \alpha(t - \text{net})$$

Gradient Descent method (First order Gradient)

- Assume $J(\mathbf{p})$ is the loss function of our learning problem in ANN.
- $\mathbf{p}$ denotes the vector which includes all tunable parameters (All weights and bias terms) and It is aimed to minimize J by optimizing $\mathbf{p}$.
$$\mathbf{p}^* = \arg \min_{\mathbf{p}} J(\mathbf{p})$$
- Assuming $J(\mathbf{p})$ is a smooth function of $\mathbf{p}$ around $\mathbf{p} := \mathbf{p}_0$, the Taylor expansion of $J(\mathbf{p})$ is defined around $\mathbf{p} := \mathbf{p}_0$ as follows:

$$J(\mathbf{p}) = J(\mathbf{p}_0) + (\mathbf{p} - \mathbf{p}_0)^T \nabla J(\mathbf{p}_0) + (\mathbf{p} - \mathbf{p}_0)^T \nabla^2 J(\mathbf{p}_0) (\mathbf{p} - \mathbf{p}_0) + h. o. t.$$

$$\text{First order Gradient: } \nabla J(\mathbf{p}_0) = \left. \frac{\partial J}{\partial \mathbf{p}} \right|_{\mathbf{p}=\mathbf{p}_0}$$

$$\text{Second order Gradient(Hessian Matrix): } \nabla^2 J(\mathbf{p}_0) = \left. \frac{\partial^2 J}{\partial \mathbf{p}^2} \right|_{\mathbf{p}=\mathbf{p}_0}$$

- Assuming that First order gradient is not zero, one can say that for a small enough variation of $\mathbf{p}$ we have:

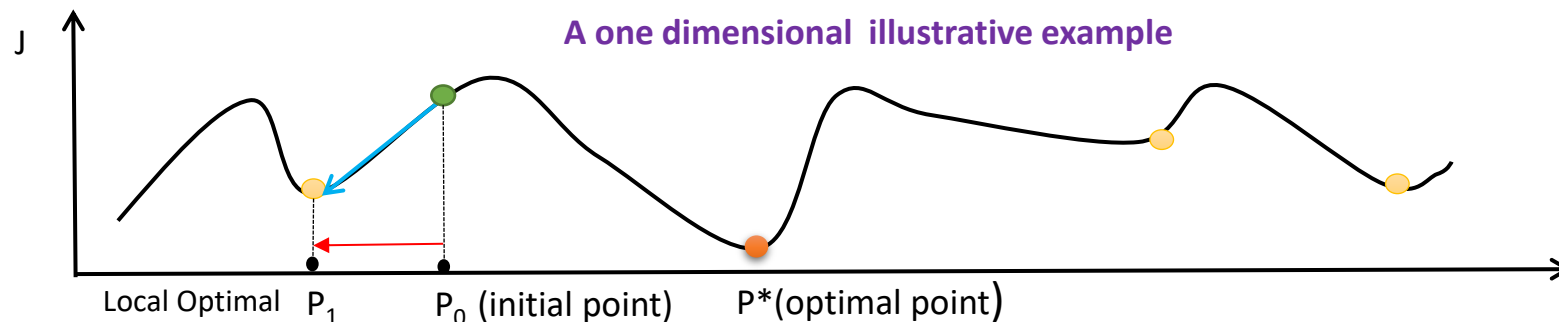
$$J(\mathbf{p}_0 + \delta\mathbf{p}) \cong J(\mathbf{p}_0) + (\delta\mathbf{p})^T \nabla J(\mathbf{p}_0) \quad \|\delta\mathbf{p}\| \rightarrow 0$$

- Defining $\delta\mathbf{p} = -\alpha \nabla J(\mathbf{p})$, and considering α as a small enough positive coefficient, we have:

$$J(\mathbf{p}_0 + \delta\mathbf{p}) \cong J(\mathbf{p}_0) - \alpha \|\nabla J(\mathbf{p}_0)\|^2 < J(\mathbf{p}_0) \quad \alpha \rightarrow 0$$

As a result, while first order gradient is not zero, the loss function will be decreased.

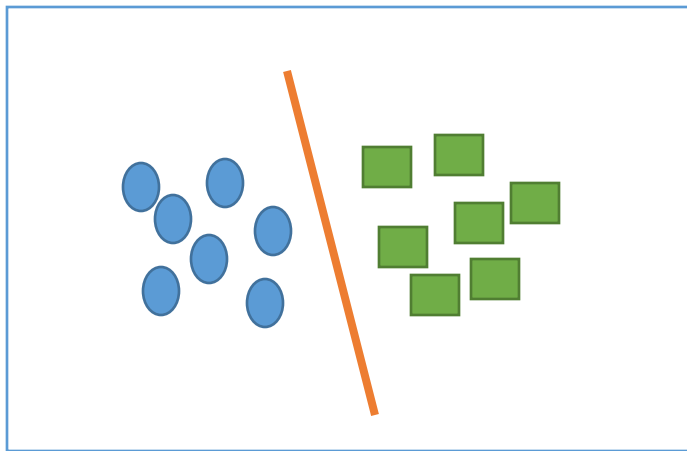
Repeating the above rule, the loss function decreasing is continued until $\mathbf{p}$ converges to a local optimal point like $\mathbf{p}_1$. if $\mathbf{p}_1$ is equal to $\mathbf{p}^*$ (the absolute optimal point) the loss function has met his absolute minimum point otherwise a sub-optimal answer in minimizing the function has been resulted.



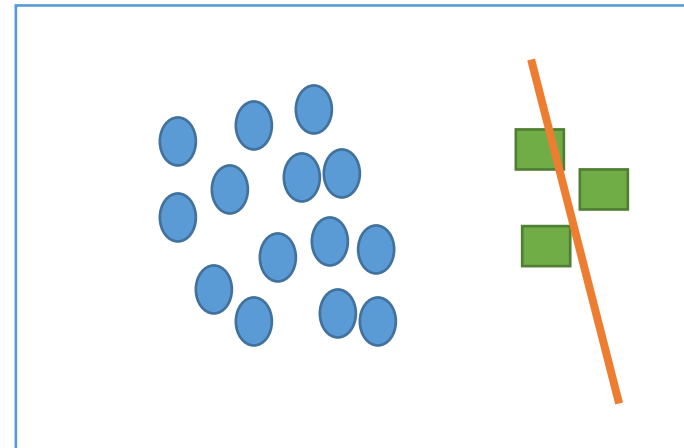
Delta Convergence Property

Since the error is defined as difference of the target and net (not difference of target and output), the net (not necessarily the output of network) becomes near to the target.

If distribution of patterns in two classes is near together in shape and population, the “Delta” will learn a robust separation hyper-plane among patterns of two classes (Figure a), otherwise, there will no guarantee for true separation (Figure b).



(a)



(b)

A solution

Replace the “sign” function with a “soft sign” such as “tanh”. Unlike “sign”, “tanh” is smooth and one can compute the new updating rules by gradient descent.

$$J_p(\mathbf{w}, b) = 0.5(t(p) - \tanh(\gamma \text{net}(x(p), w, b)))^2$$

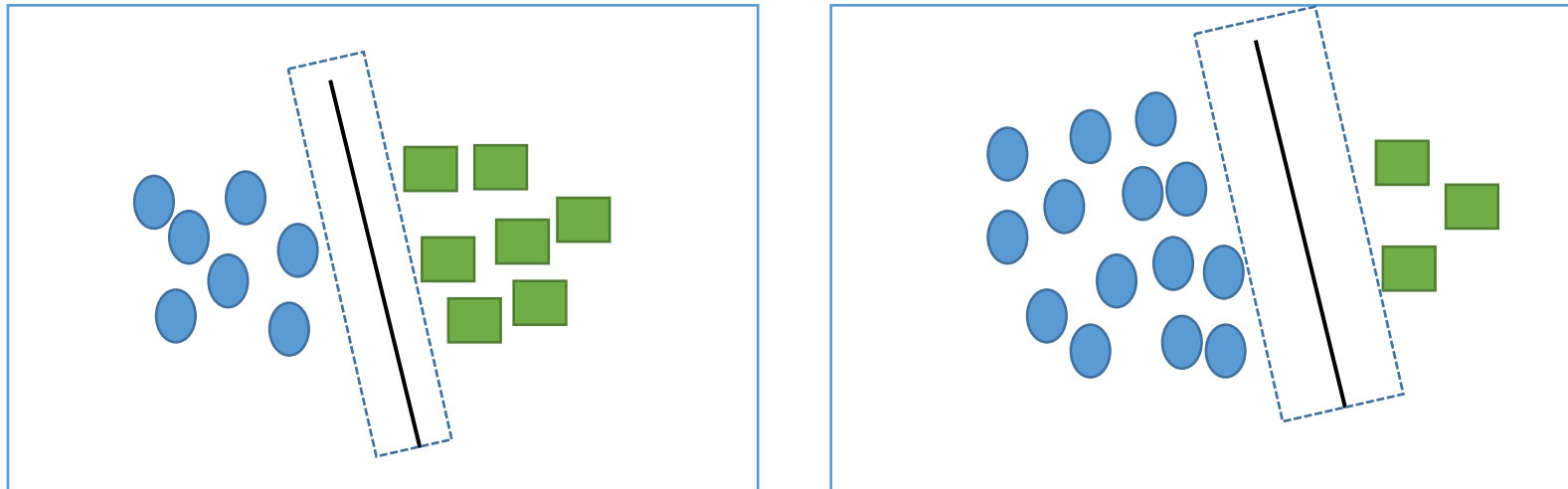
$$\delta w_i = w_i^+ - w_i^- = -\alpha \frac{\partial J_p(\mathbf{w}, b)}{\partial w_i} \Big|_{w^-, b^-} = \alpha(1 - h^2)\gamma(t - h)x_i$$

$$\delta b = b^+ - b^- = -\alpha \frac{\partial J_p(\mathbf{w}, b)}{\partial b} \Big|_{w^-, b^-} = \alpha(1 - h^2)\gamma(t - h)$$

if γ is chosen so big then “tanh” will be near to a “sign” function. Now, by decreasing the loss function, the output of network will converge to the target value (as it was done in perceptron network).

Using the aforementioned solution

1. By applying the aforementioned solution, independent from the shape and populations of patterns, the partitioning line will be converged between patterns of two classes with a large enough margins.
2. This modified version of Delta learning rule has superiority to the perception network which was stopped by finding every partitioning line.



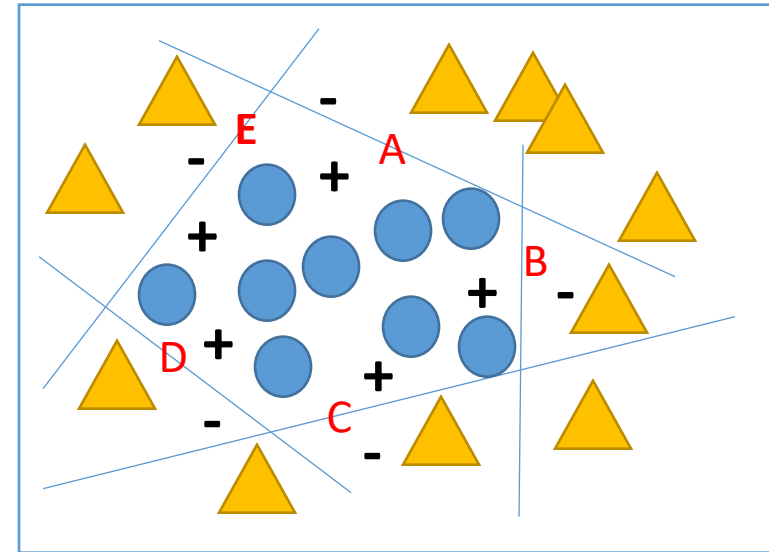
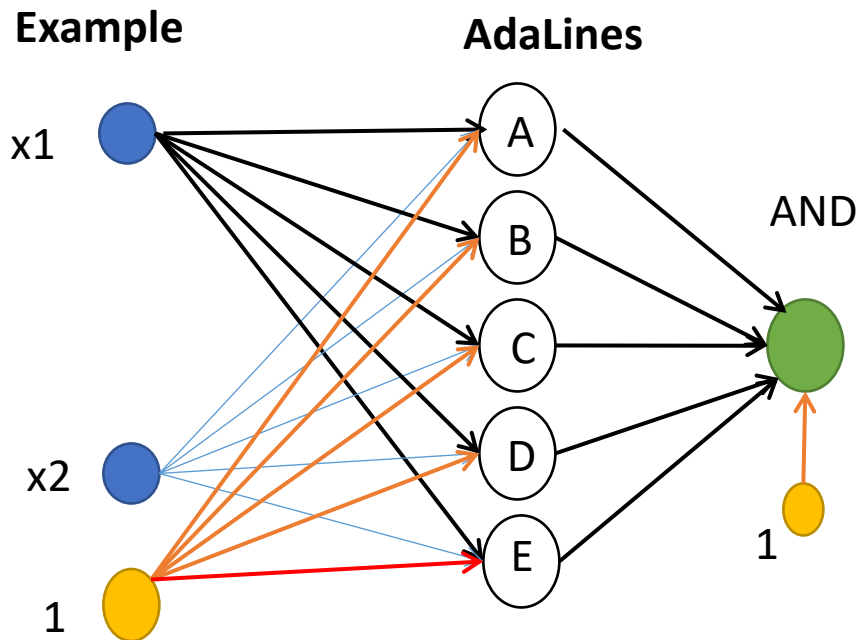
Chapter 2 – part 1

Multi AdaLine (MAdaLine)

Multi AdaLine (MAdaLine)

MAdaLine is made from some AdaLines which are combined by an “OR” or “AND” M&P neuron.

MAdaLine has the required potential to separate a set of patterns (with a unique label) surrounded by a convex hyper-polygon

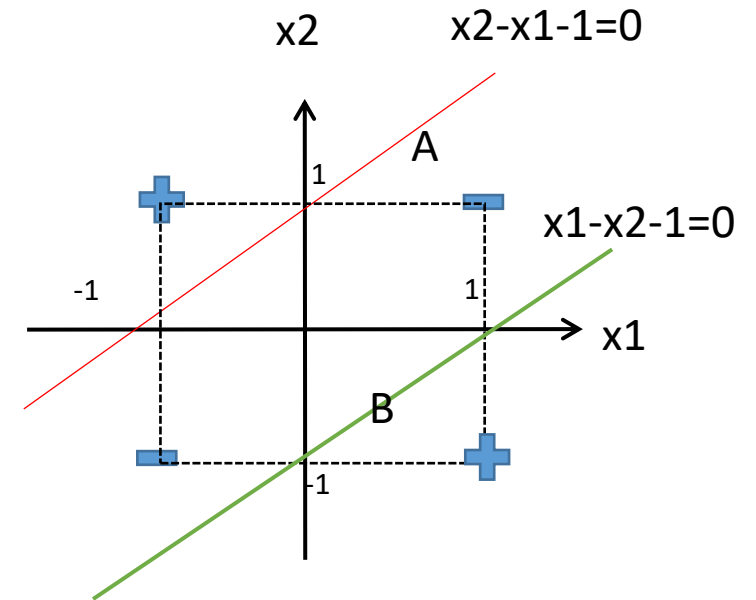
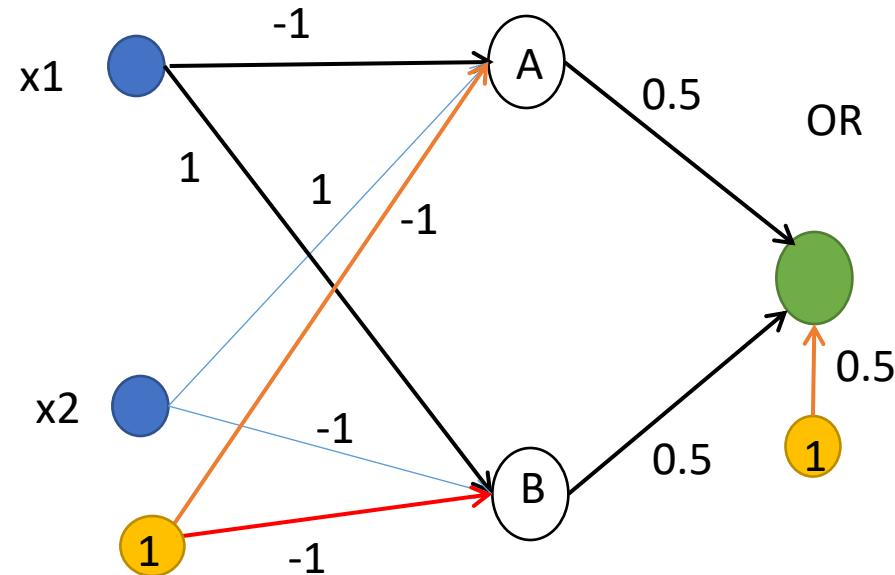


Learning the required number of Adalines and their weights, to make an appropriate convex hyper polygon, is not straightforward

XOR Example (Widrow & Hoff 1960)

1- Classification of “Xor” logic by MAdaLine

n=2				
sample	s		1	t
	x1	x2		
1	1	1	1	-1
2	1	-1	1	1
3	-1	1	1	1
4	-1	-1	1	-1



Two Algorithms:

MRI: The output neuron is adjusted as an “OR” logic and only the weights and biases of hidden neurons are trained.

MRII: All the weights and biases of hidden and output layers are trained.

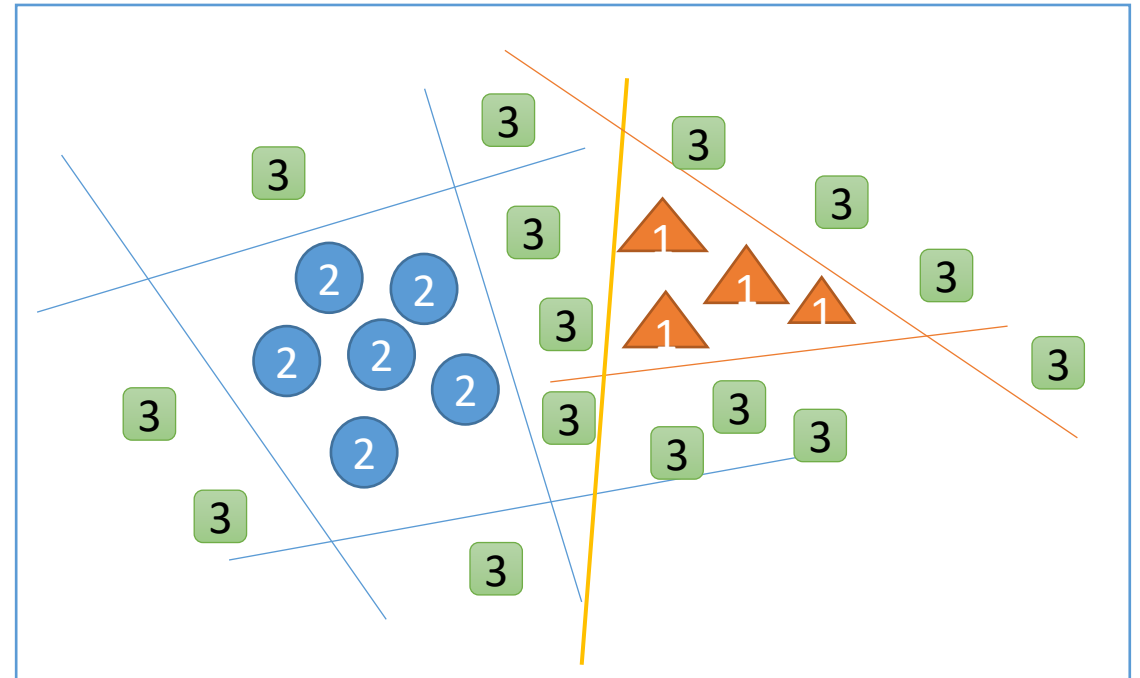
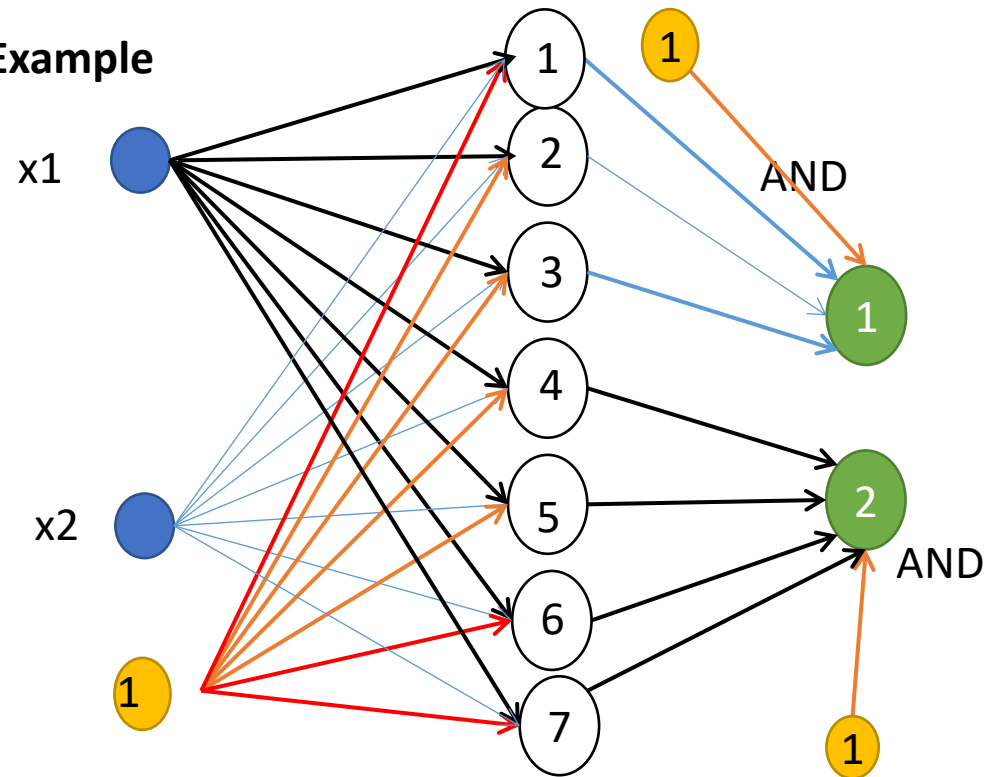
Two above algorithms have been explained in the book.

MAdaLine can separate more than one set of patterns surrounded by convex-hyper polygons.

AdaLines

An illustrative example

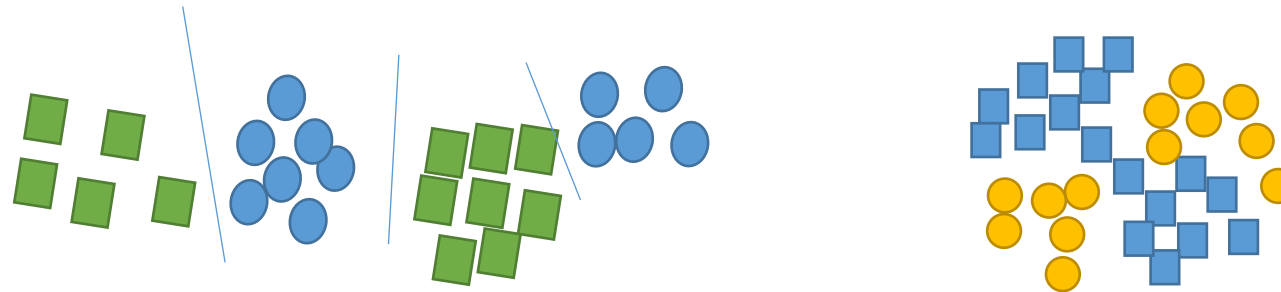
Example



Some hidden neurons with near hyper-planes can be merged and the redundancy is decreased.

Disabilities of MAdaLine

1. There is no systematic/straightforward optimization method to update the weights in MAdaLine. Also, **MAdaLine** can not make analogue outputs for **regression Problems**.
2. MAdaLine does not work in following cases:
 - The patterns of a certain class are divided to more than one convex hyper-polygon shape cluster.
 - The patterns of a certain class have a non-convex hyper-polygon shape cluster.



3. Redundancies including **correlations** and disturbances can not be removed/Filtered by MAdaLine networks.

Solutions for aforementioned problems

1- Using gradient search methods → **Multi Layer Perceptron with one hidden layer**

(1): Systematic optimization method / Convex shape partitioning/ Mapping

2- Considering two hidden layers in the network → **Multi Layer Perceptron with two hidden layers**

(1)+(2): Non-Convex shape partitioning/ Mapping

3-1. Using Encoding Part (auto-encoder/stacked RBMs) with two hidden layers → **Multi Layer Neural Networks**

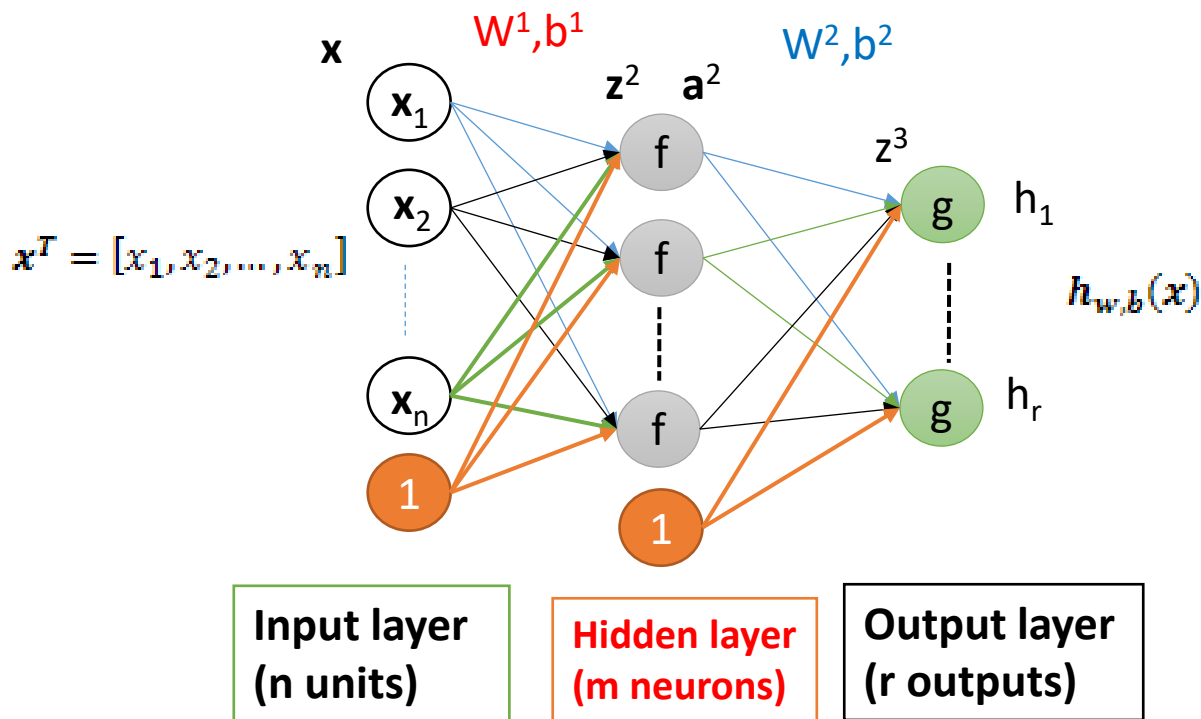
(1)+(2)+(3): Dimensionality Reduction

3-2. Using some Filters and scaling layers → **Convolutional Neural Network**

(1)+(2)+(3)+(4): disturbance Filtering

Multi-Layer Perceptron (MLP) with one hidden layer

Multi Layer Perceptron with one hidden layer (Rumelhart 1986)



$$W^1 = [w_{ji}^1], \quad i = 1, \dots, n \quad j = 1 \dots m$$

$$b^1 = [b_j^1], \quad j = 1 \dots m$$

$$W^2 = [w_{kj}^2], \quad j = 1, \dots, m \quad k = 1, \dots, r$$

$$b^2 = [b_k^2], \quad k = 1 \dots r$$

Bias Vectors: $\mathbf{b} = \{\mathbf{b}^1, \mathbf{b}^2\}$

Weight Matrices: $\mathbf{W} = \{\mathbf{w}^1, \mathbf{w}^2\}$

Input-output Equation:

$$h_{w,b}(x) = g \left(W^2 f \left(\underbrace{W^1 x + b^1}_{\substack{z^2 \\ a^2}} \right) + b^2 \right)$$

Activation Functions $f(x) = sig(x) = \frac{1}{1 + \exp(-x)}$ or $\tanh(x)$

$$g(x) = \begin{cases} sig(x) & \text{Classification} \\ x & \text{Regression} \end{cases}$$

Learning Process of MLP with one hidden layer

Error Back Propagation (BP)

Training Set: $\{x^q, y^q\}_{q=1}^Q$ Labels (Clas.) - Analogue outputs(Reg.)

(W, b) are tuned in order to $h_{w,b}(x^q) \rightarrow y^q$

Loss Function(SSE): $J(\mathbf{w}, \mathbf{b}) = 0.5 \sum_{q=1}^Q \|h_{\mathbf{w},\mathbf{b}}(\mathbf{x}) - \mathbf{y}^q\|^2$

Updating Rules for sample “q” and after “call” stage and assuming the output is scalar
(from the “output-layer” in a back-ward manner to the “first-layer”):

$$w_{kj}^2(new) = w_{kj}^2(old) - \alpha \left. \frac{\partial J}{\partial w_{kj}^2} \right|_{(w,b)(old)} \quad \frac{\partial J}{\partial w_{kj}^2} = -(h_k(x) - y_k^q) \dot{g}(z_k^3) a_j^2 \quad \alpha > 0: \text{Learning rate}$$

$$b_k^2(new) = b_k^2(old) - \alpha \left. \frac{\partial J}{\partial b_k^2} \right|_{(w,b)} \quad \frac{\partial J}{\partial b_k^2} = -(h_k(x) - y_k^q) \dot{g}(z_k^3)$$

$$w_{ji}^1(new) = w_{ji}^1(old) - \alpha \left. \frac{\partial J}{\partial w_{ji}^1} \right|_{(w,b)(old)} \quad \frac{\partial J}{\partial w_{ji}^1} = - \left(\sum_{k=1}^r (h_k(x) - y_k) \dot{g}(z_k^3) w_{kj}^2 \right) \dot{f}(z_j^2) x_i$$

$$b_j^1(new) = b_j^1(old) - \alpha \left. \frac{\partial J}{\partial b_j^1} \right|_{(w,b)(old)} \quad \frac{\partial J}{\partial b_j^1} = - \left(\sum_{k=1}^r (h_k(x) - y_k) \dot{g}(z_k^3) w_{kj}^2 \right) \dot{f}(z_j^2)$$

Learning Algorithm for MLP

- 0- Scale every input between “0 up to 1” or “-1 up to 1” (to get better results in learning) .
- 1- Initialize the weights and biases as small random real values. Consider a small learning rate to guarantee a successful local search and then set **epoch=0**.
- 2- Apply input(s) of training set to the Network in one of following methods.
 - 2-1 Batch based : All data points are applied, simultaneously .
 - 2-2 Stochastic based: Data are shuffled and then data points are applied one by one.
 - 2-3 Stochastic mini batch based: training data set are randomly divided to a certain number of subsets (min-batches); then at each time a min-batch is chosen and applied.
- 3- According to the considered method(2-1, 2-2 or 2-3), compute the **outputs** of the network, the **error** and the **loss function**(Call stage)
- 4- According to the considered method(2-1, 2-2 or 2-3), use the updating rules for the weights and biases (ReCall stage)
- 5- If the method 2-2 or 2-3 has been chosen, repeat steps 3 and 4 until all training data points or min-batches are applied, completely.
- 6- if the loss function is decreased continually for both training and validation sets, let **epoch:=epoch+1** and return to 2, else go to the next step.
- 7- The algorithm is terminated. To check the generalization of the network it is evaluated by the considered validation dataset.

Some Important Notes

1. How many neurons should be defined in the hidden layer ?

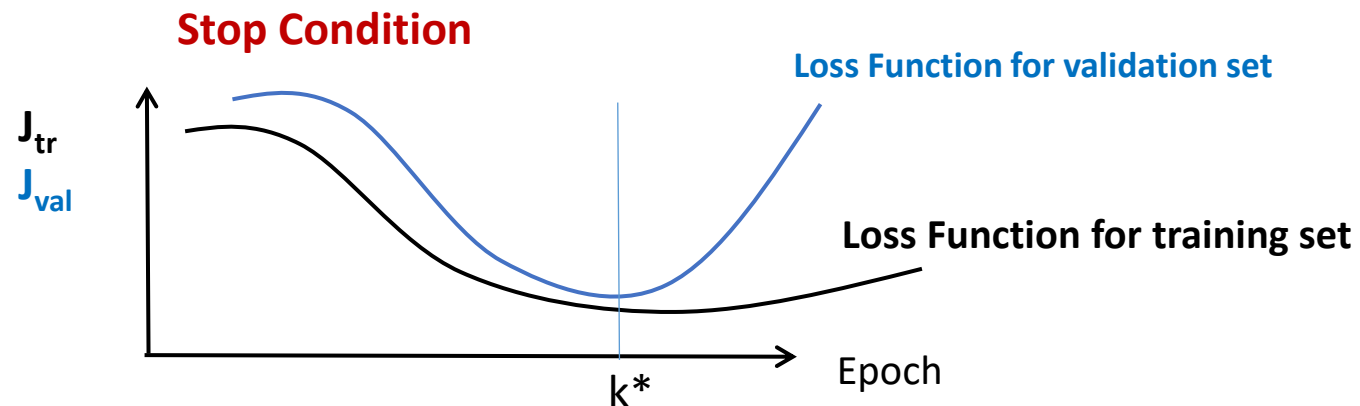
It depends on the required complexity in partitioning (for classification) or mapping (for regression). It is suggested that the appropriate number of neurons optimized from low to up

2. To avoid diverging the weights to infinity, which is an invalid learning causing over-fitness, it is suggested to add a regularization term in loss function. ($\lambda \geq 0$: regulation weight)

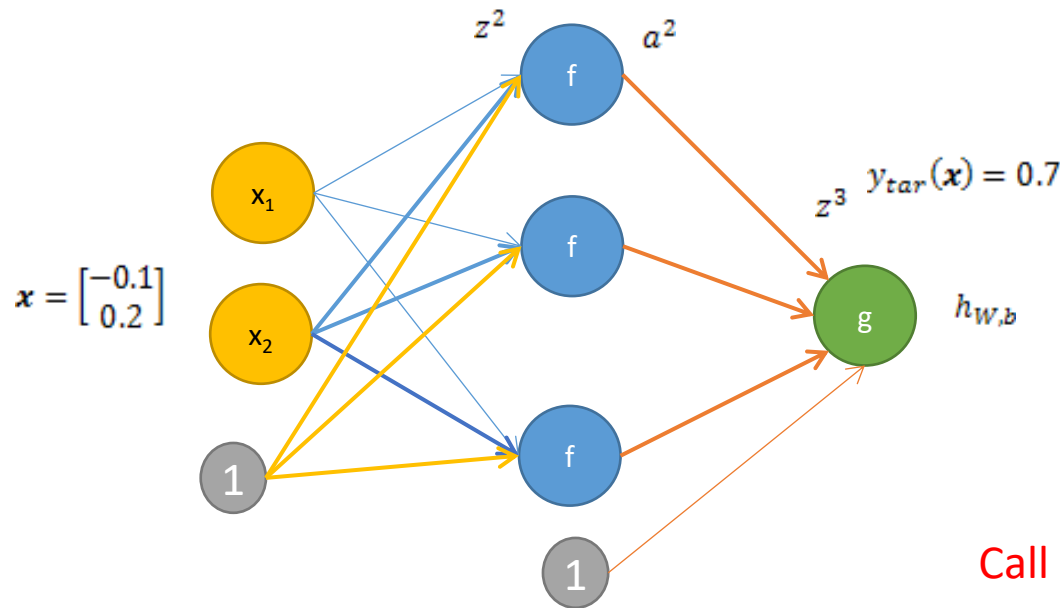
$$J(\mathbf{w}, \mathbf{b}) = 0.5 \sum_{q=1}^Q \|\mathbf{h}_{\mathbf{w}, \mathbf{b}}(\mathbf{x}^q) - \mathbf{y}^q\|^2 + \lambda \|\mathbf{w}\|^2$$

3. The stop condition become satisfied when the computed loss function of the validation set does not decrease any more.

Although decreasing rate of loss function for training set is important, to avoid over fitness, the decreasing rate of loss function for validation set should be considered for stop condition.



A Simple Example of Call and Recall in an MLP with one hidden layer



$$W^1 = [w_{ji}^1] = \begin{bmatrix} 1 & 2 \\ -1 & 1 \\ 2 & 0 \end{bmatrix} \quad \mathbf{b}^1 = [b_j^1] = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

$$W^2 = [w_{kj}^2] = [-1 \quad 1 \quad 1] \quad \mathbf{b}^2 = [b_k^2] = 0.5$$

$$i = 1, 2 \quad j = 1, 2, 3 \quad k = 1$$

$$f : \tanh \quad g : \tanh$$

$$h_{W,b}(\mathbf{x}) = g(W^2 f(W^1 \mathbf{x} + \mathbf{b}^1) + b^2) \quad z^2 = \begin{bmatrix} 1.3 \\ 0.5 \\ -1.2 \end{bmatrix} \quad a^2 = \begin{bmatrix} 0.86 \\ 0.29 \\ -0.83 \end{bmatrix}$$

$$J_{W,b}(\mathbf{x}) = 0.5(h_{W,b}(\mathbf{x}) - y(\mathbf{x}))^2 \quad z^3 = -0.90 \quad h_{w,b}(\mathbf{x}) = -0.72$$

$$J = 1.0057$$

ReCall

$$w_{kj}^+ = w_{kj}^- - \alpha \frac{\partial J}{\partial w_{kj}} \bigg|_{W^-, b^-} \quad b_k^+ = b_k^- - \alpha \frac{\partial J}{\partial b_k} \bigg|_{W^-, b^-} \quad W^1 = \begin{bmatrix} 0.9998 & 2.004 \\ -1.0006 & 1.0013 \\ 1.9998 & 0.0004 \end{bmatrix} \quad \mathbf{b}^1 = \begin{bmatrix} 0.5916 \\ 0.2000 \\ -0.5724 \end{bmatrix}$$

$$w_{ji}^+ = w_{ji}^- - \alpha \frac{\partial J}{\partial w_{ji}} \bigg|_{W^-, b^-} \quad b_j^+ = b_j^- - \alpha \frac{\partial J}{\partial b_j} \bigg|_{W^-, b^-} \quad W^2 = [-0.99 \quad 1.002 \quad 0.9943] \quad b^2 = 0.5069$$

$$\alpha = 0.01$$

$$h_{w,b}(\mathbf{x}) = -0.7067 \quad J = 0.9894$$

*Loss function is decreased

Derivative Chain Rule

$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial h} \frac{\partial h}{\partial z_k^3} \frac{\partial z_k^3}{\partial w_{kj}} = (h - y) g'(z_k^3) a_j^2$$

$$\frac{\partial J}{\partial b_k} = \frac{\partial J}{\partial h} \frac{\partial h}{\partial z_k^3} \frac{\partial z_k^3}{\partial b_k} = (h - y) g'(z_k^3)$$

$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial h} \left(\sum_{h=1}^k \frac{\partial h}{\partial z_h^3} \frac{\partial z_h^3}{\partial a_j} \right) \frac{\partial a_j}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}} = (h - y) \left(\sum_{h=1}^k g'(z_k^3) w_{kj} \right) f'(z_j^2) x_i$$

$$\frac{\partial J}{\partial b_j} = \frac{\partial J}{\partial h} \left(\sum_{h=1}^k \frac{\partial h}{\partial z_h^3} \frac{\partial z_h^3}{\partial a_j} \right) \frac{\partial a_j}{\partial z_j} \frac{\partial z_j}{\partial b_j} = (h - y) \left(\sum_{h=1}^k g'(z_k^3) w_{kj} \right) f'(z_j^2)$$

$$f = \tanh \quad f' = (1 - f^2)$$

$$g = \tanh \quad g' = (1 - g^2)$$

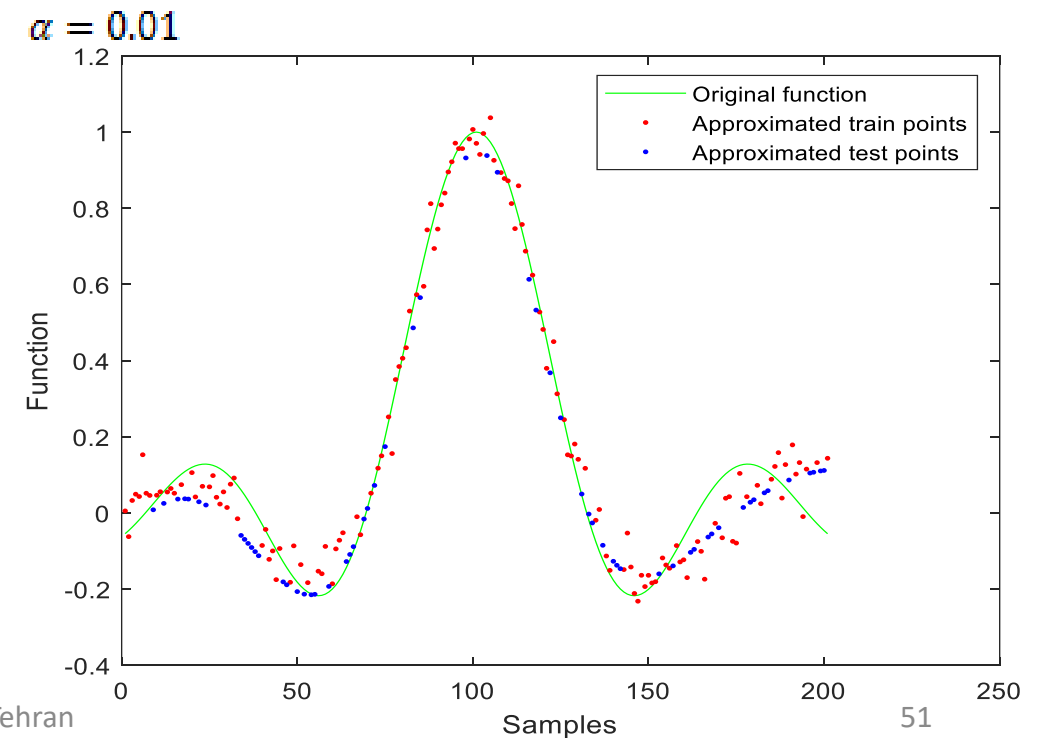
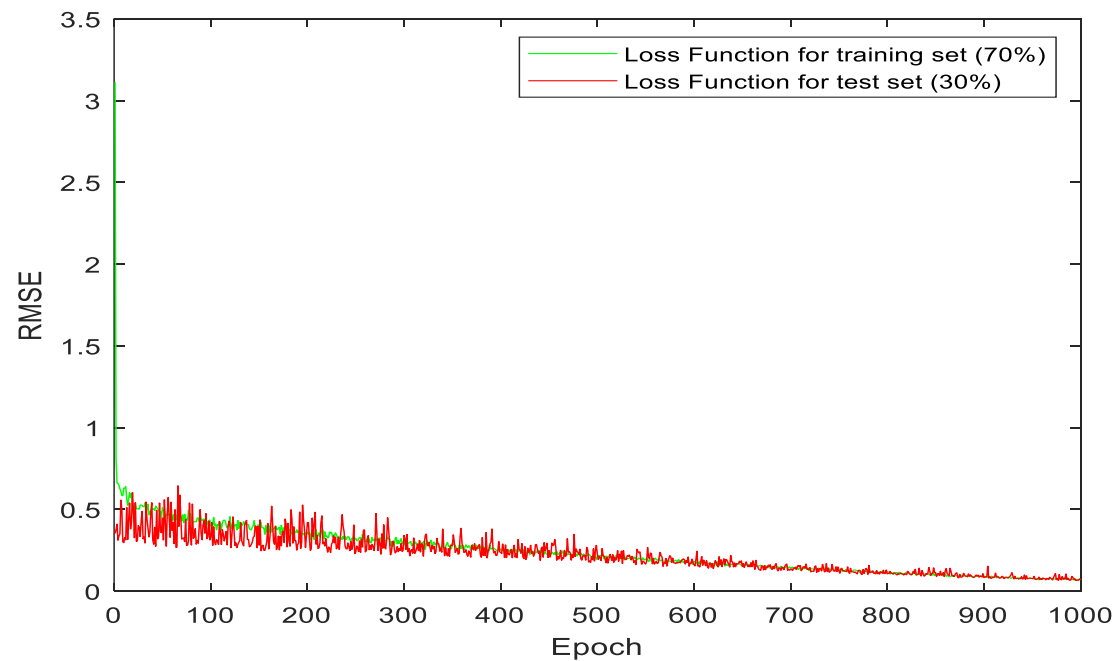
Some notes

1. The gradient rules for any parameter can be computed independently
2. Since some parts of updating rules for parameters in further layers can be saved in backward layers too, in order to reduce the computation load, it is suggested to compute gradients of parameters from further layers to first layers.
3. Since the derivative of sigmoid function is less than one, one can say the norm of gradients for first layers is reduced significantly (gradient vanishing).

illustrative Example1: Function Approximation

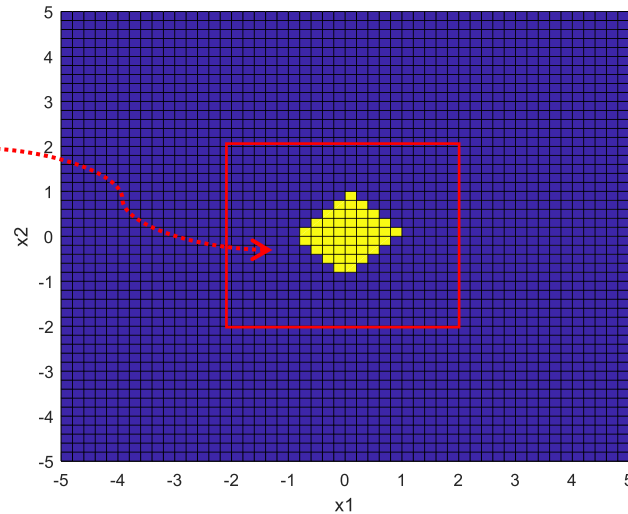
Function $\text{Sinc}(x) = \sin(x)/x$

- **Dimension:** $n=1$ input / $m=100$ hidden neurons / $r=1$ output
- **Training Set:** 140 pairs / **Validation Set:** 60 pair
- **BP strategy:** Stochastic point based/ number of epochs=1000
- **Loss Function:** Root Mean Square Error (RMSE)/ learning rate:



Illustrative Example2: Classification(Convex Shape)

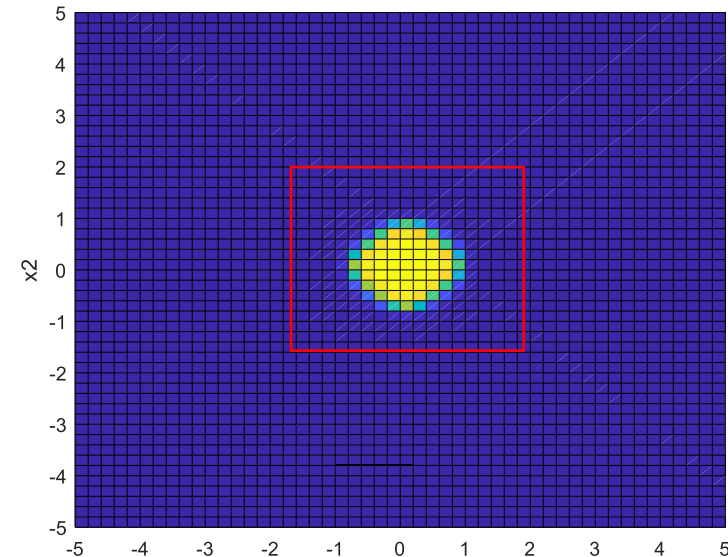
Target Partitions



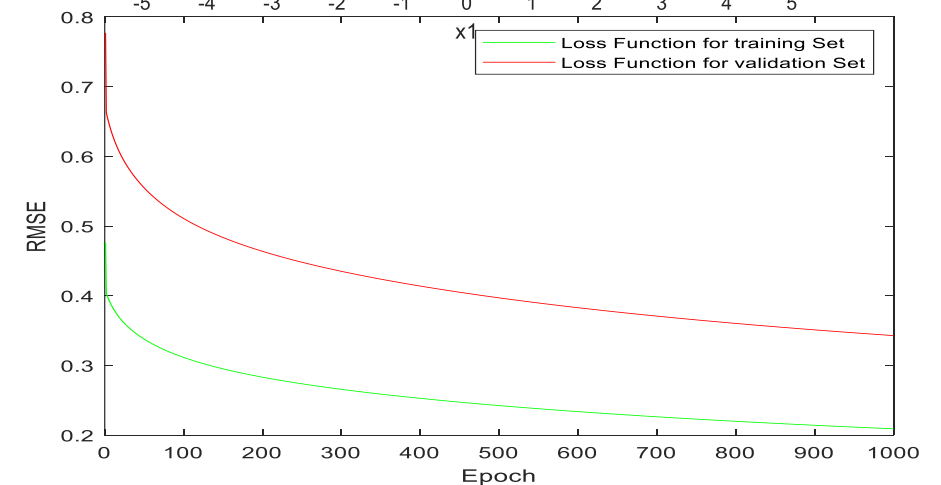
Training set

After Training

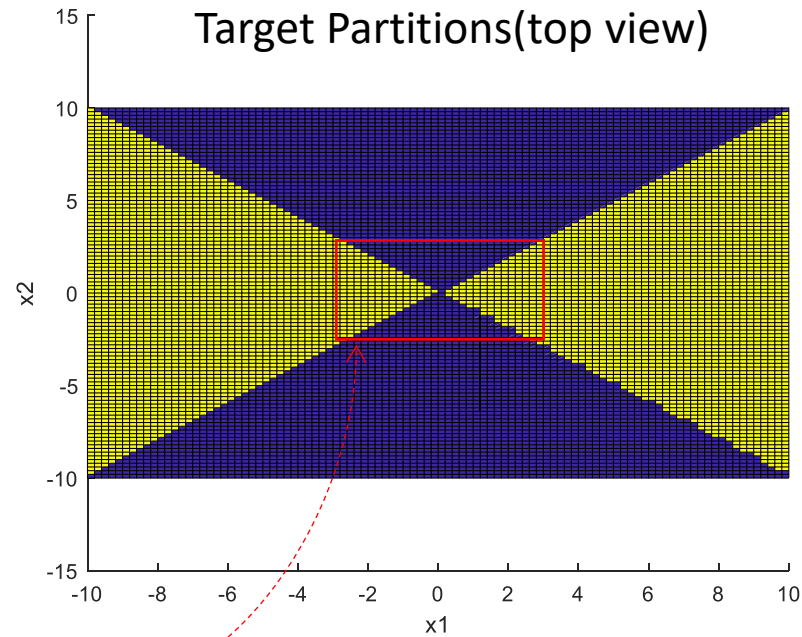
Trained Partitions



N. of Neurons in Layer1=4
Learning rate=0.001
Learning Method: Batch-Based BP
N. of Training samples=100
N. of validation samples=400



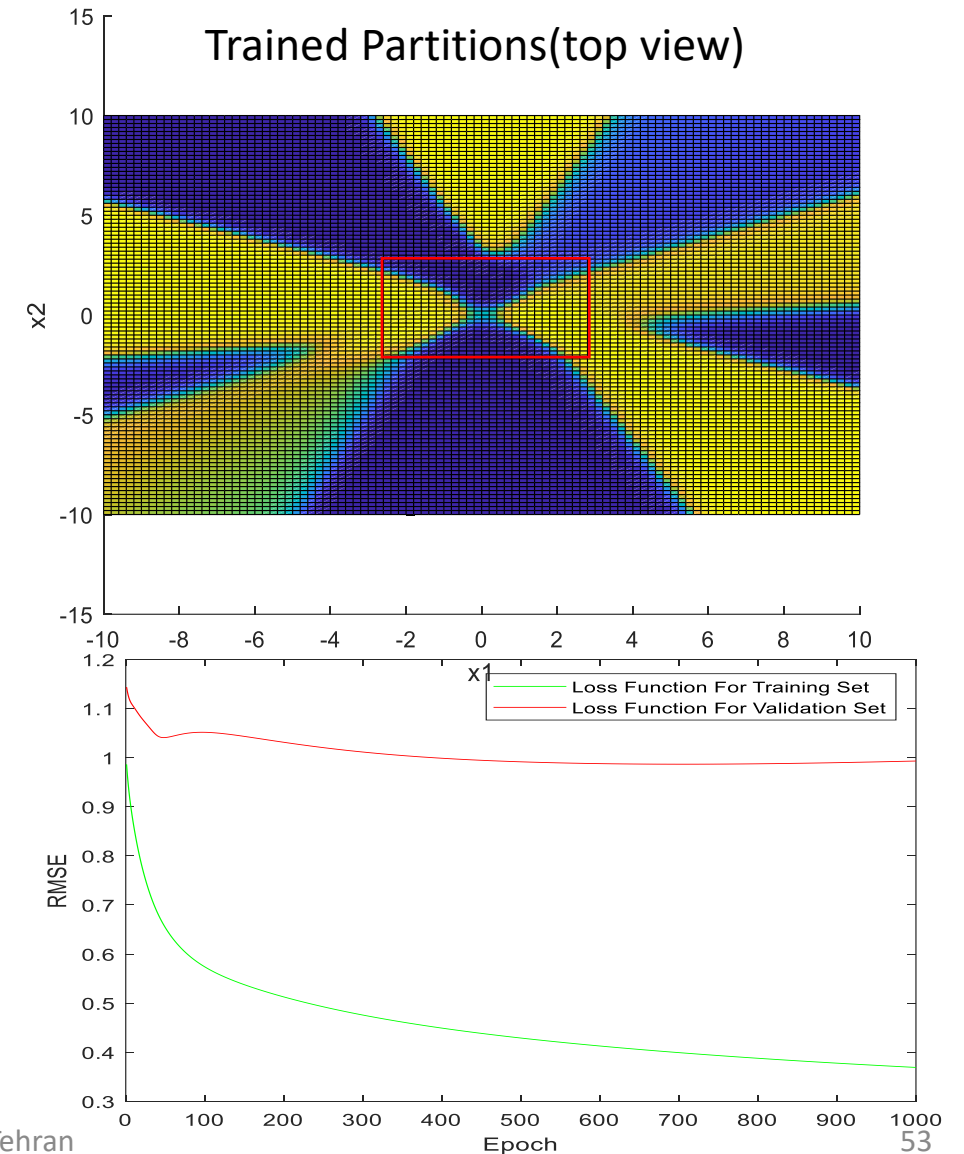
Illustrative Example3: Classification (Non-convex Shape)



Training set

N. of Neurons in Layer1=10
Learning rate=0.005
N. of training set=100 samples
N. of validation set=400 samples
Learning set: Batch-Based BP

After Training



Potential of MLP with one hidden layer in Function Approximation

1. MLP with one hidden layer is a general function approximator

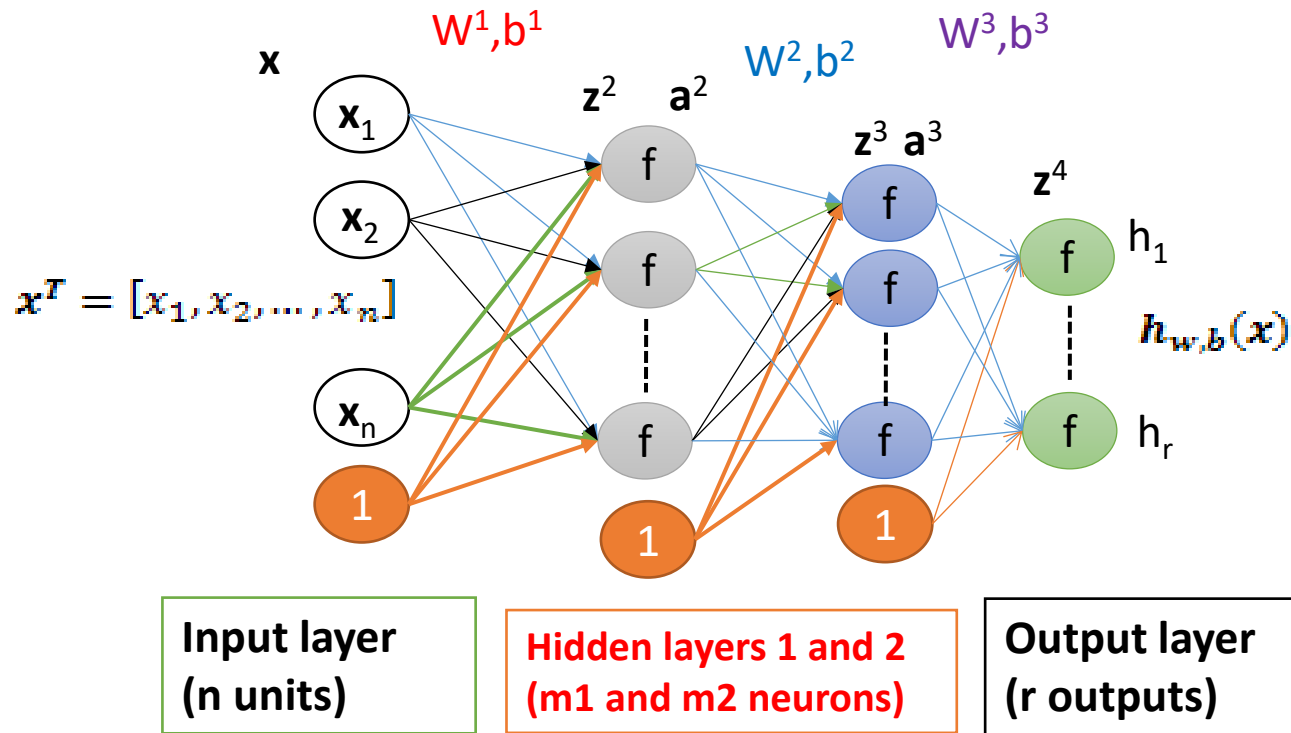
It was proved that: a MLP NN with one hidden layer containing a finite number of neurons can approximate any continuous functions on compact subsets of $\mathbb{R}^n$, Cybenko, 1989. It does not touch upon the algorithmic learnability of those parameters.

2. Though it was mentioned that MAdaLine can not model the family of non-convex functions, due to using softness of $\tanh()$ and using not quantized weights in output layer, non-convex regions (maps) can be modeled by MLP with one hidden layer.

3. In such cases, the required redundancy in using hidden neurons is high and hence over fitness will be occurred and the generalization of the network particularly for uncompact subsets (unbounded maps or partitions) is not guaranteed.

Multi-Layer Perceptron (MLP) with two hidden layers

Multi Layer Perceptron with two hidden layers



$$W^1 = [w_{ji}^1], i = 1, \dots, n \quad j = 1, \dots, m_1$$

$$W^2 = [w_{kj}^2], k = 1, \dots, m_2$$

$$W^3 = [w_{hk}^3], h = 1, \dots, r$$

$$b^1 = [b_j^1], b^2 = [b_k^2] \text{ and } b^3 = [b_h^3]$$

$$\mathbf{W} = \{W^1, W^2, W^3\}$$

$$\mathbf{b} = \{b^1, b^2, b^3\}$$

$$h_{w,b}(x) = g(W^3 f \left(\underbrace{W^2 f \left(\underbrace{W^1 x + b^1}_{z^2} \right) + b^2}_{a^2} \right) + b^3)_{z^4}$$

Learning Process of MLP with two hidden layers

Error Back Propagation (BP)

Training Set: $\{x^q, y^q\}_{q=1}^Q$ Labels (Clas.) - Analogue outputs (Reg.)

(W, b) is optimized in order to $\mathbf{h}_{w,b}(x^q) \rightarrow y^q$

Loss Function(SSE): $J(\mathbf{w}, \mathbf{b}) = 0.5 \sum_{q=1}^Q \|\mathbf{h}_{\mathbf{w},\mathbf{b}}(\mathbf{x}) - y^q\|^2$

- Updating Rules (from output-layer back-ward to first-layer):

Using error back-propagation, all weight matrices as well as bias vectors should be updated.

$$\mathbf{W}(new) = \mathbf{W}(old) - \alpha \frac{\partial J}{\partial \mathbf{W}_{(\mathbf{w},\mathbf{b})}(old)} \quad \alpha > 0: \text{Learning rate}$$

$$\mathbf{b}(new) = \mathbf{b}(old) - \alpha \frac{\partial J}{\partial \mathbf{b}_{(\mathbf{w},\mathbf{b})}(old)}$$

Potential of the MLP with two hidden layers

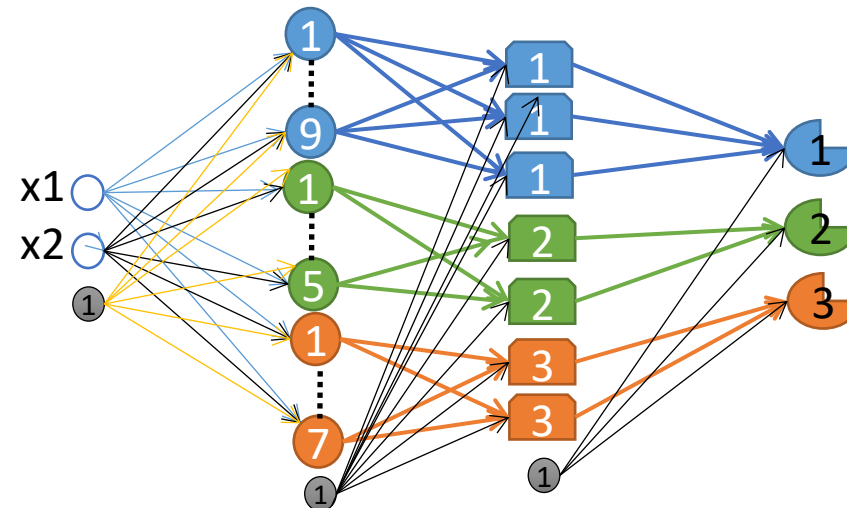
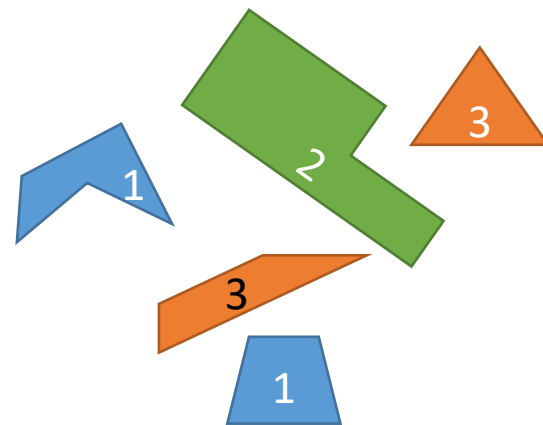
1- Partitioning (Classification)

- Considering that the input space is the same feature space, a MLP Network with two hidden layers has the required potential in (reliable) classification of any number of classes with any shape and with any accuracy.

Reliable classification means: (1) the redundancy in number of the required neurons is low (2), the robustness against input uncertainties is high and (3) the generalization for un-compact subsets is high.

- However, it is not known that how many neurons in the hidden layers are required. Also, the weight matrices or bias vectors are not determined for such purposes.

An Illustrative Example:



Potential of the MLP with two hidden layers

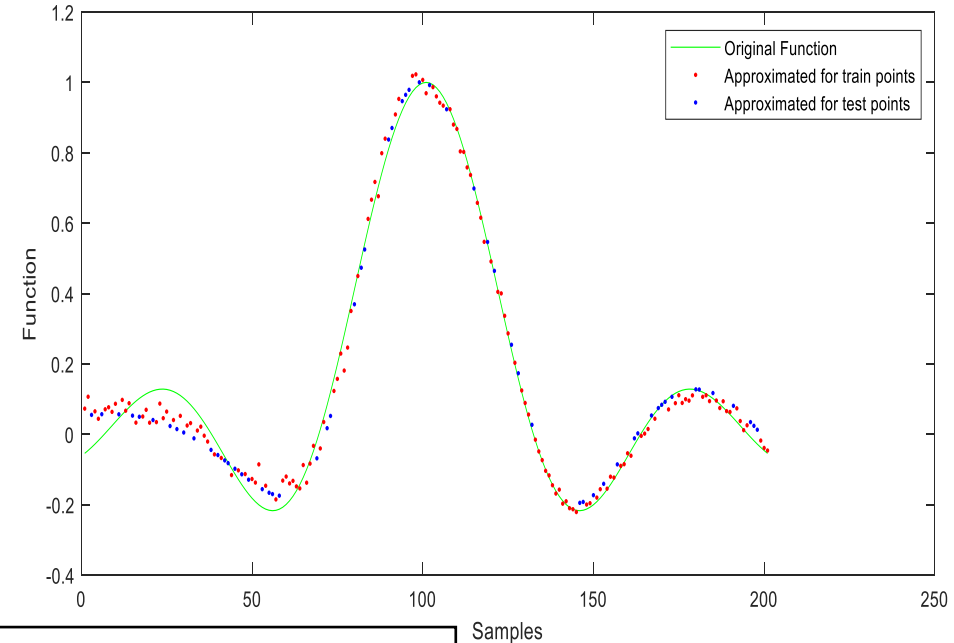
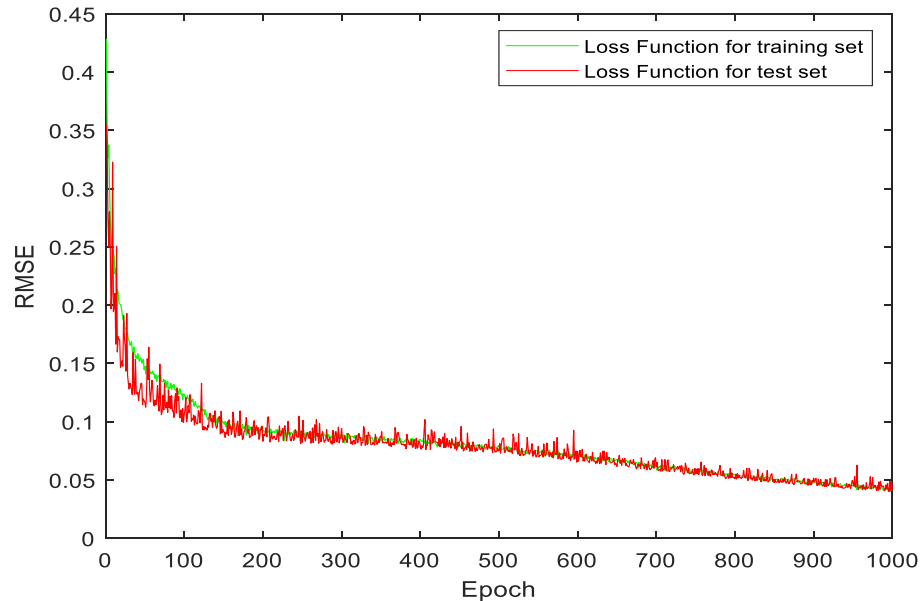
2- Mapping(Regression)

- An MLP with two hidden layers can be used to build any map with any required accuracy .
- **Since a Regression problem can be approximated by a classification problem,** By increasing the number of classes, an arbitrary smooth map can be approximated and hence any map can be build by any required accuracy.
- However, it is not actually known that how many neurons (in hidden layers) are required ;also, the weight matrices or bias vectors are not known for such purposes.

Illustrative Example1: Function Approximation

Function : $\text{Sinc}(x) = \sin(x)/x$

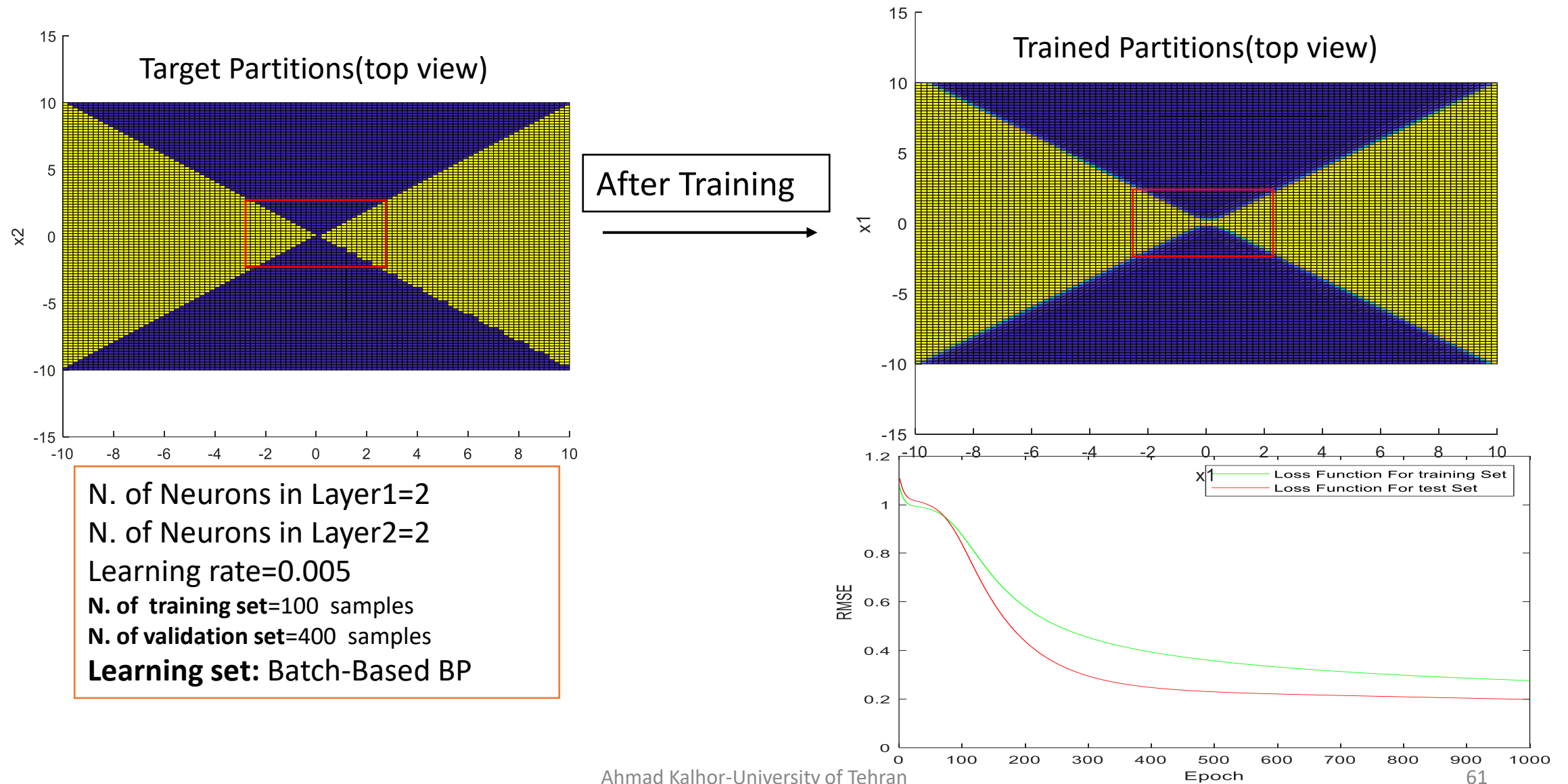
- **Dimension:** $n=1$ inputs / $m_1=5$ first hidden neurons / $m_2=5$ second hidden neurons / $r=1$ output
- **Training Set/BP strategy/Loss Function/ learning rate** (The same used in MLP with one hidden layer)



Comparison with MLP including one hidden-layer

1. Less number of parameters **2. better accuracy** **3. better validation**

Illustrative Example2: Classification

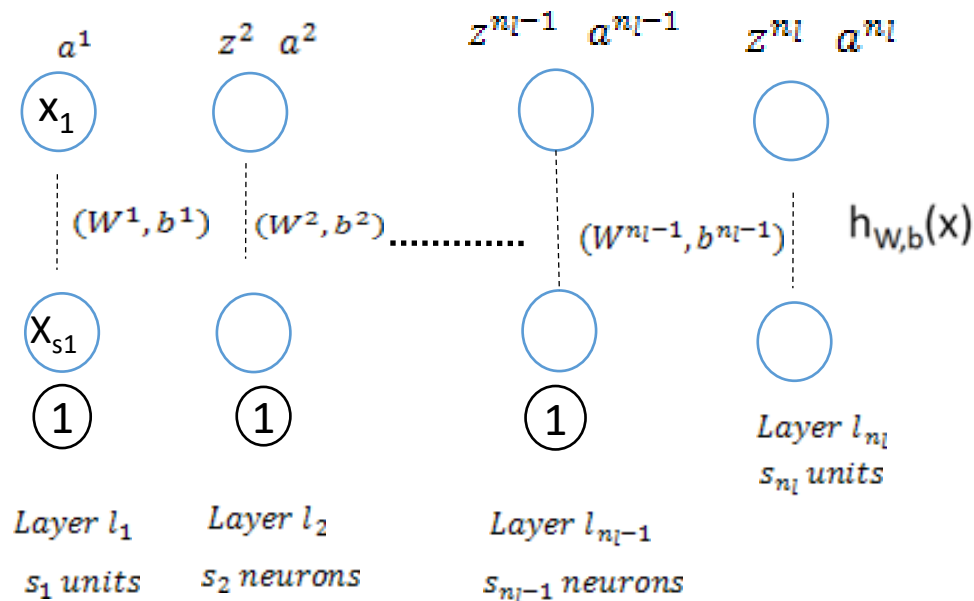


Multi Layer Neural Networks (MLNNs)

Multi Layer Neural Networks (MLNNs)

(<http://ufldl.stanford.edu/tutorial/supervised/MultiLayerNeuralNetworks/>)

- MLNN is a feed-forward network including arbitrary number of (fully-connected or partially connected) layers, in which, each layer contains an arbitrary number of neurons.
- MLNNs with more than one hidden layer can be considered as Deep NNs.
- Applying extra layers allows Feature Extraction: (1) Data dimension-reduction, and (2) Data Filtering and scaling.
- Some known architectures of MLNNs are **Auto-encodes**, **Deep-Belief Neural Networks** and **Convolutional Neural Networks**, which will be introduced and explained later.
- At first, the (supervised) learning algorithm is explained.



$$a_i^{(1)} = x_i$$

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$$

$$a^{(l+1)} = f(z^{(l+1)})$$

$$l = 1, 2, \dots, n_l - 1$$

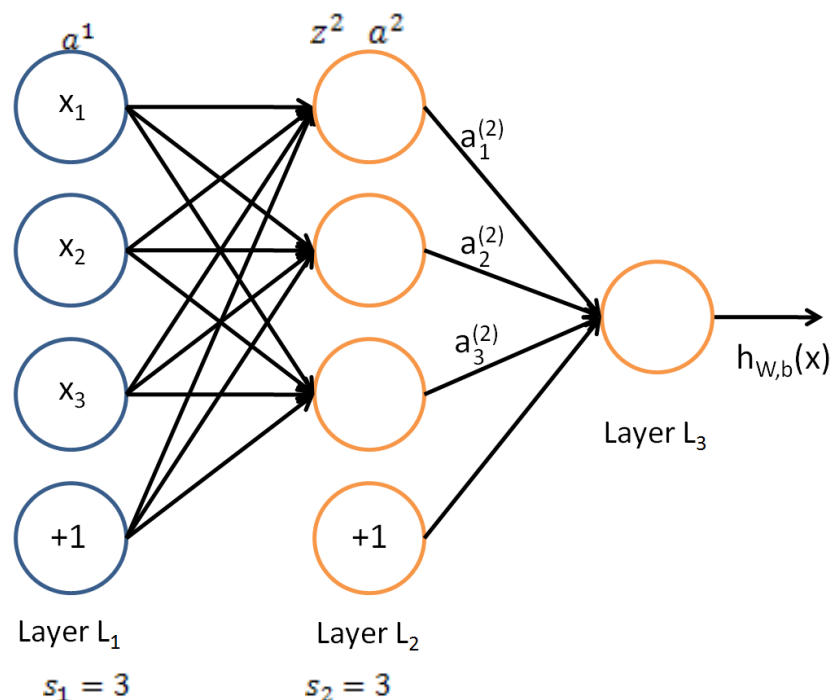
$$h_{W,b}(x) = a^{n_l}$$

$$(W, b) = \{(W^1, b^1), \dots, (W^{n_l-1}, b^{n_l-1})\}$$

m training examples.

training set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

A Simple Example (to introduce the notation)



$$(W, b) = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}) \quad n_l = 3$$

$$a_1^{(2)} = f(W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 + W_{13}^{(1)} x_3 + b_1^{(1)})$$

$$a_2^{(2)} = f(W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 + W_{23}^{(1)} x_3 + b_2^{(1)})$$

$$a_3^{(2)} = f(W_{31}^{(1)} x_1 + W_{32}^{(1)} x_2 + W_{33}^{(1)} x_3 + b_3^{(1)})$$

$$h_{W,b}(x) = a_1^{(3)} = f(W_{11}^{(2)} a_1^{(2)} + W_{12}^{(2)} a_2^{(2)} + W_{13}^{(2)} a_3^{(2)} + b_1^{(2)})$$

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)],$$

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)} \quad l = 1, 2$$

$$a^{(l+1)} = f(z^{(l+1)})$$

MLNN Learning Algorithm

Backpropagation Algorithm

Suppose we have a fixed training set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$ of m training examples. We can train our neural network using batch gradient descent. In detail, for a single training example (x, y) , we define the cost function with respect to that single example to be:

$$J(W, b; x, y) = \frac{1}{2} \|h_{W,b}(x) - y\|^2.$$

This is a (one-half) squared-error cost function. Given a training set of m examples, we then define the overall cost function to be:

$$\begin{aligned} J(W, b) &= \left[\frac{1}{m} \sum_{i=1}^m J(W, b; x^{(i)}, y^{(i)}) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2 \\ &= \left[\frac{1}{m} \sum_{i=1}^m \left(\frac{1}{2} \|h_{W,b}(x^{(i)}) - y^{(i)}\|^2 \right) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2 \end{aligned}$$

One iteration of gradient descent updates the parameters W, b as follows:

$$\begin{aligned} W_{ij}^{(l)} &= W_{ij}^{(l)} - \alpha \frac{\partial}{\partial W_{ij}^{(l)}} J(W, b) \\ b_i^{(l)} &= b_i^{(l)} - \alpha \frac{\partial}{\partial b_i^{(l)}} J(W, b) \end{aligned}$$

Back propagation Algorithm in Detail

In detail, here is the backpropagation algorithm:

1. Perform a feedforward pass, computing the activations for layers L_2, L_3 , and so on up to the output layer L_{n_l} .

2. For each output unit i in layer n_l (the output layer), set

$$\delta_i^{(n_l)} = \frac{\partial}{\partial z_i^{(n_l)}} \frac{1}{2} \|y - h_{W,b}(x)\|^2 = -(y_i - a_i^{(n_l)}) \cdot f'(z_i^{(n_l)})$$

3. For $l = n_l - 1, n_l - 2, n_l - 3, \dots, 2$

For each node i in layer l , set

$$\delta_i^{(l)} = \left(\sum_{j=1}^{s_{l+1}} W_{ji}^{(l)} \delta_j^{(l+1)} \right) f'(z_i^{(l)})$$

4. Compute the desired partial derivatives, which are given as:

$$\begin{aligned} \frac{\partial}{\partial W_{ij}^{(l)}} J(W, b; x, y) &= a_j^{(l)} \delta_i^{(l+1)} \\ \frac{\partial}{\partial b_i^{(l)}} J(W, b; x, y) &= \delta_i^{(l+1)}. \end{aligned}$$

The BP algorithm by matrix-vector notation

“ $\bullet$ ” to denote the element-wise product operator

The algorithm can then be written:

1. Perform a feedforward pass, computing the activations for layers L_2, L_3 , up to the output layer L_{n_l} , using the equations defining the forward propagation steps

2. For the output layer (layer n_l), set

$$\delta^{(n_l)} = -(y - a^{(n_l)}) \bullet f'(z^{(n_l)})$$

3. For $l = n_l - 1, n_l - 2, n_l - 3, \dots, 2$, set

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \bullet f'(z^{(l)})$$

4. Compute the desired partial derivatives:

$$\begin{aligned}\nabla_{W^{(l)}} J(W, b; x, y) &= \delta^{(l+1)} (a^{(l)})^T, \\ \nabla_{b^{(l)}} J(W, b; x, y) &= \delta^{(l+1)}.\end{aligned}$$

For sigmoid function: $f'(z_i^{(l)}) = a_i^{(l)} (1 - a_i^{(l)})$

Full Gradient Descent Algorithm

Finally, we are ready to describe the full gradient descent algorithm. In the pseudo-code below, $\Delta W^{(l)}$ is a matrix (of the same dimension as $W^{(l)}$), and $\Delta b^{(l)}$ is a vector (of the same dimension as $b^{(l)}$). Note that in this notation, " $\Delta W^{(l)}$ " is a matrix, and in particular it isn't " Δ times $W^{(l)}$." We implement one iteration of batch gradient descent as follows:

1. Set $\Delta W^{(l)} := 0$, $\Delta b^{(l)} := 0$ (matrix/vector of zeros) for all l .
2. For $i = 1$ to m ,
 1. Use backpropagation to compute $\nabla_{W^{(l)}} J(W, b; x, y)$ and $\nabla_{b^{(l)}} J(W, b; x, y)$.
 2. Set $\Delta W^{(l)} := \Delta W^{(l)} + \nabla_{W^{(l)}} J(W, b; x, y)$.
 3. Set $\Delta b^{(l)} := \Delta b^{(l)} + \nabla_{b^{(l)}} J(W, b; x, y)$.
3. Update the parameters:

$$W^{(l)} = W^{(l)} - \alpha \left[\left(\frac{1}{m} \Delta W^{(l)} \right) + \lambda W^{(l)} \right]$$
$$b^{(l)} = b^{(l)} - \alpha \left[\frac{1}{m} \Delta b^{(l)} \right]$$

*Former Introduced MLP with one or two hidden layers can be considered as MLNN with three or four layers.

Dimensionality Reduction in MLNNs

Dimensionality Reduction in MLNNs

- In many real-world high dimensional patterns there are correlations among dimensions.
- Existing correlations among inputs avoids reliable and generalized partitioning and mapping in classification and regression problems.
- It is desired that by removing correlations, the input data vector(matrix) is transformed to a space with lower dimension.

$$\mathbf{x} \in \mathbb{R}^n \xrightarrow{\text{Dimension reduction}} \mathbf{z} = Tr(\mathbf{x}) \in \mathbb{R}^m \quad (m < n)$$

$$\mathbf{x} \in \mathbb{R}^{n_1 \times n_2} \xrightarrow{\text{Dimension reduction}} \mathbf{z} = Tr(\mathbf{x}) \in \mathbb{R}^{m_1 \times m_2} \quad (m_1 \times m_2 < n_1 \times n_2)$$

- Since only correlations are removed, no valuable information is reduced, and the new space(with lower dimension) is enough to retrieve the initial input data.

$$\mathbf{z} \in \mathbb{R}^{m_1 \times m_2} \xrightarrow{\text{Retrive initial data}} \mathbf{x} = Tr^{-1}(\mathbf{z}) \in \mathbb{R}^{n_1 \times n_2}$$

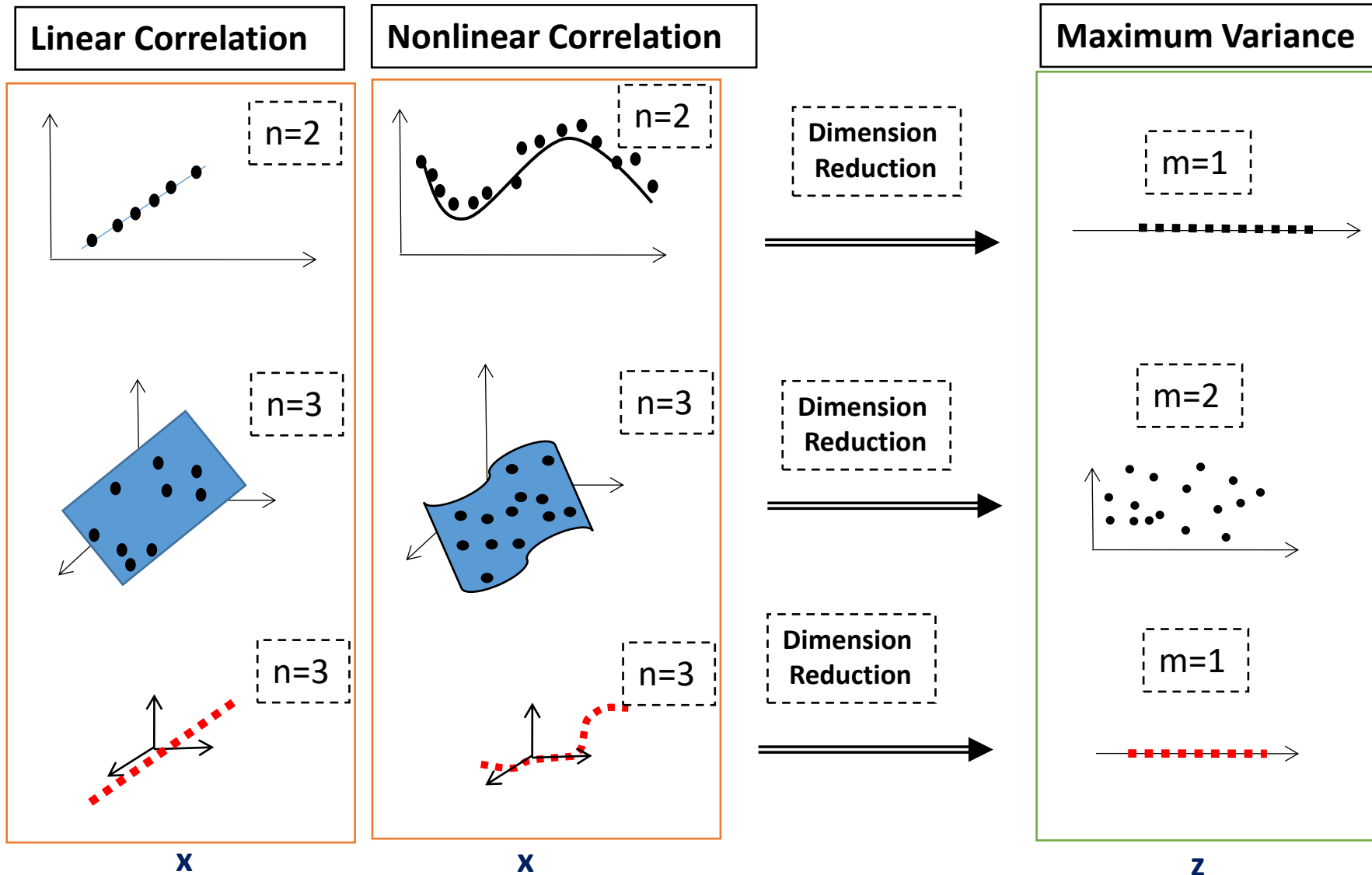
$$\mathbf{z} \in \mathbb{R}^m \xrightarrow{\text{Retrive initial data}} \mathbf{x} = Tr^{-1}(\mathbf{z}) \in \mathbb{R}^n$$

- Some initial layers of a MLNN can be employed for Dimensionality Reduction purposes.

Important Notes

1. If there are linear or nonlinear correlations among input dimensions, by removing correlations one can will get a new feature space (with lower dimensions), in which features are independent.
2. It is possible to retrieve original input-data by using inverse of the transformation.
3. In geometric view: if (n-dimensional) input data points are distributed on a (m-dimensional) hyper-surface where $m < n$, there will be (n-m) independent correlations among dimensions (nullity space).
4. By Removing correlations among input dimensions, nullity spaces of the input data will be removed and the variance of the feature space will be maximized.

Illustrative Examples



Removing Linear Correlations by PCA

Assume $\mathbf{x}$ denotes an n -dimensional point of the input space: $\mathbf{x}^T = [1, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]$ (bias has been augmented)

There will be a linear correlation among different input dimensions, if for all samples of $\mathbf{x}$ there is a non-zero real scalar (the normal vector), $\mathbf{a}^T = [\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_n]$, in order that:

$$\mathbf{a}^T \mathbf{x} = 0, \quad \mathbf{x} \in \{\mathbf{x}^i\}_{i=1}^m$$

One can show for r independent linear correlations, “ r ” Eigen values of the “Auto Correlation Matrix” (R) are zero and their corresponding Eigen vectors reveal “ r ” independent linear correlations.

$$R = \sum_{i=1}^m \mathbf{x}^i \mathbf{x}^{iT} = V \Lambda V^T$$

$$\Lambda = \begin{bmatrix} \Lambda^1 & 0 \\ 0 & \Lambda^2 \end{bmatrix} \quad \Lambda^1 = \text{diag}([\lambda_1, \dots, \lambda_r]) = \mathbf{0} \quad \Lambda^2 = \text{diag}([\lambda_{r+1}, \dots, \lambda_n])$$

$$V = \begin{bmatrix} \underbrace{\mathbf{v}_1 \dots \mathbf{v}_r}_{V_1} & \underbrace{\mathbf{v}_{r+1} \dots \mathbf{v}_n}_{V_2} \end{bmatrix} = [V_1 \ V_2]$$

r independent Correlations: $\mathbf{v}_j^T \mathbf{x} = 0 (j \in \{1, \dots, r\})$

Dimensionality Reduction and Data Retrieving for Linear Correlations

$$V^T x = \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix} x = \begin{bmatrix} V_1^T x \\ V_2^T x \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{r \times 1} \\ \mathbf{z}_{(n-r) \times 1} \end{bmatrix}$$

Nullity part which will be removed.

Informative part which forms the new space

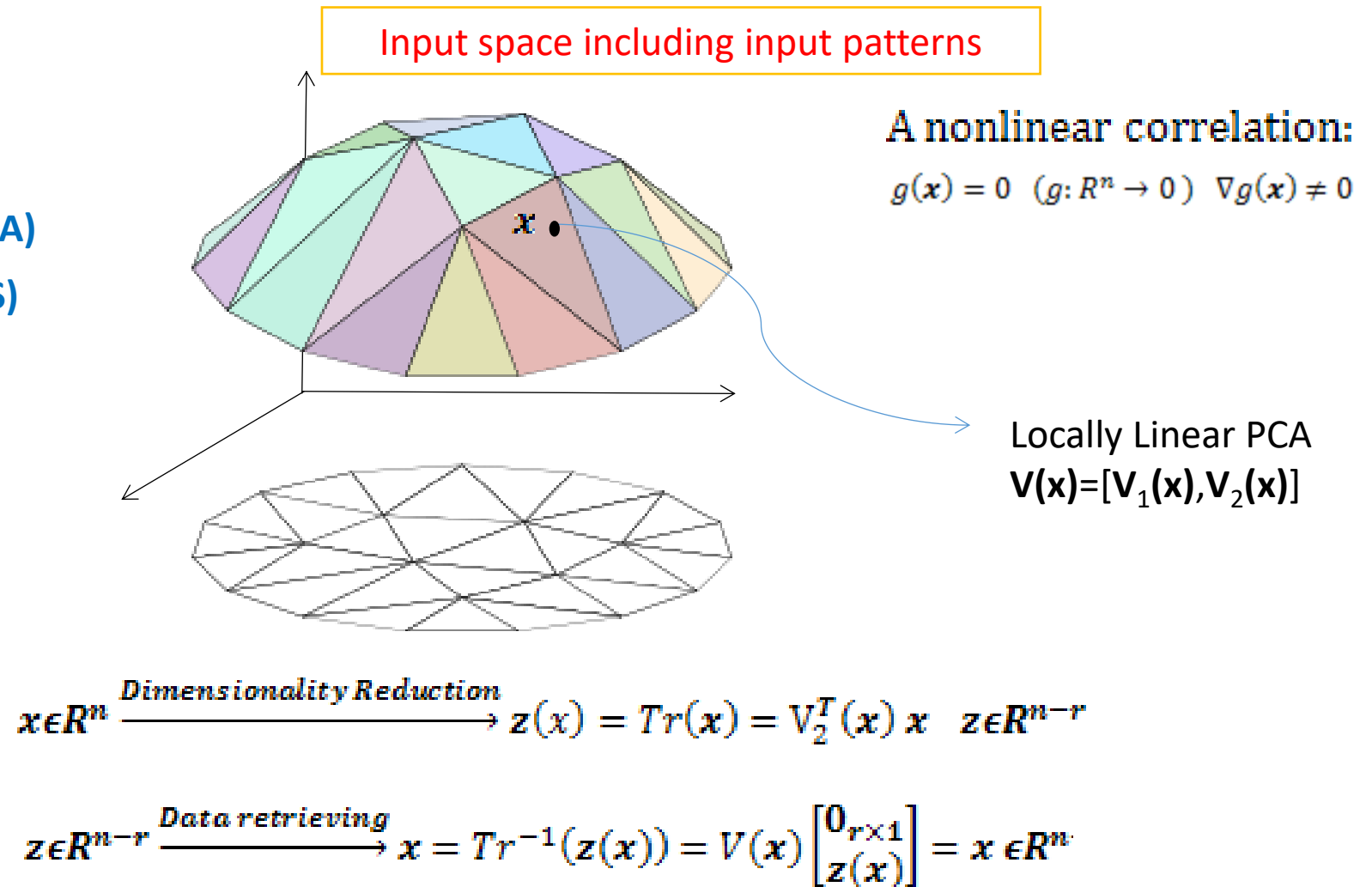
$$x \in R^n \xrightarrow{\text{Dimensionality Reduction}} z(x) = Tr(x) = V_2^T x \quad z \in R^{n-r}$$

$$z \in R^{n-r} \xrightarrow{\text{Data retrieving}} x = Tr^{-1}(z) = V \begin{bmatrix} \mathbf{0}_{r \times 1} \\ z \end{bmatrix} = VV^T x = x \in R^n$$

Nonlinear Correlation Removing

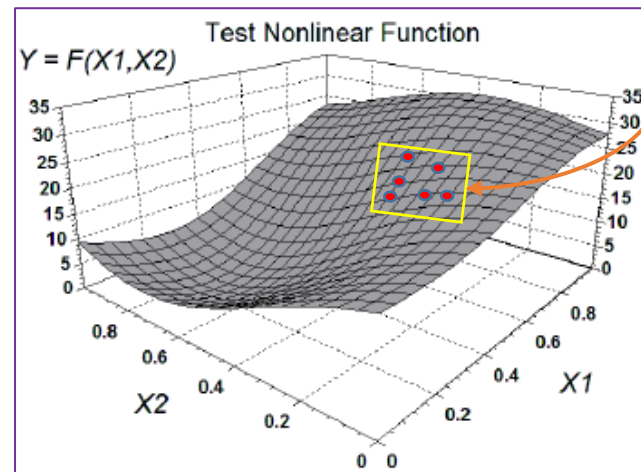
- Nonlinear PCA

1. Kernel PCA
1. Linear Discriminant Analysis(LDA)
2. Multidimensional Scaling (MDS)
3. Isomap
4. Locally Linear Embedding (LLE)
5. Laplacian Eigen map
6. Local Tangent
7. Space Alignment (LTSA)
8. Fast map



Two Valuable Notes

1. If data points, which are belonged to a limited patch of the input space, are collected, using simple PCA one will find the dimension of the nullity space.



Patch with limited size

Locally Linear PCA
 $V(x)=[V_1(x), V_2(x)]$
 $r=\dim(V_1(x))$

2. A MLNN with suitable number of layers and neurons can be used to approximate the nonlinear transformation required to remove the nonlinear correlations and dimensionality reduction.

$$x \in R^n \xrightarrow{\text{Dimensionality Reduction}} z(x) = Tr(x) = V_2^T(x) x \quad z \in R^{n-r}$$

Auto-Encoders

Auto-Encoders (Liou and Huang 2008)



Auto-Encoder

1. Auto-Encoders are a variant of MLNNs which are trained in order to remove nonlinear correlations among dimensions of the input data space (**Encoding part**).
2. To reach the aim of Auto-Encoders in Encoding, an auxiliary supervised learning process is required. Actually Auto Encoders in second part (**Decoding part**) try to represent the initially taken patterns in the output layer.
3. Auto-Encoders can be learned by the algorithm explained for MLNN. Error Back propagation is utilized to update weights and biases.

Some important notes about Auto-Encoders

1. One can say that AutoEncoders are a variant of “identity function” because:

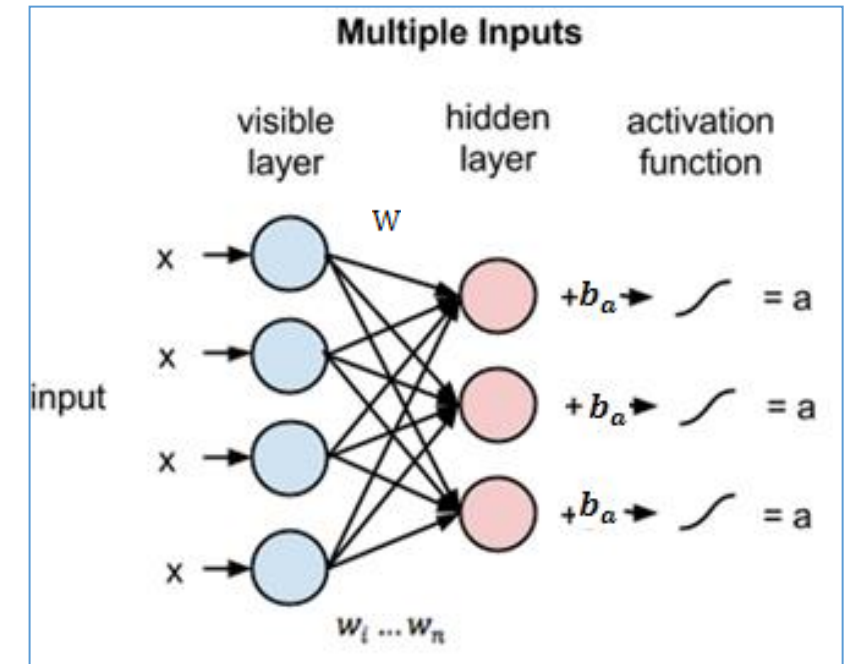
$$x = Tr^{-1}(Tr(x)) \quad x \xrightarrow{(Encoding: Tr(x))} (z) \xrightarrow{(Decoding: Tr^{-1}(z))} x$$

2. The maximum possible Dimensionality reduction of the input space is equal to the dimension of the nullity space (or the number of all independent correlations).
3. It is tried to reach to minimum possible dimension of the data space (z) in Encoding part (First Part).
4. To remove more complex nonlinear correlations, one should use more layers (depth) and neurons.
5. It is not necessary that the number of neurons in Encoding part(Decoding Part) is decreased (increased) uniformly layer by layer. It is expected that at the end of the Encoding(Decoding) part the number of neurons has been recued (increased) in comparison to the initial dimension of the input data (the decreased dimension of the data).
6. If the distribution of the input data is near to the white noise distribution, the nullity space and possible correlations will be vanished(there is no possibility in dimension reduction)
7. The nullity space can also be interpreted as the space of disturbance patterns which have occupied the nullity space with rather low repetition or density (infrequent)

Restricted Boltzmann Machine (RBM)

Restricted Boltzmann Machine(RBM) [Paul Smolensky](#) in 1986

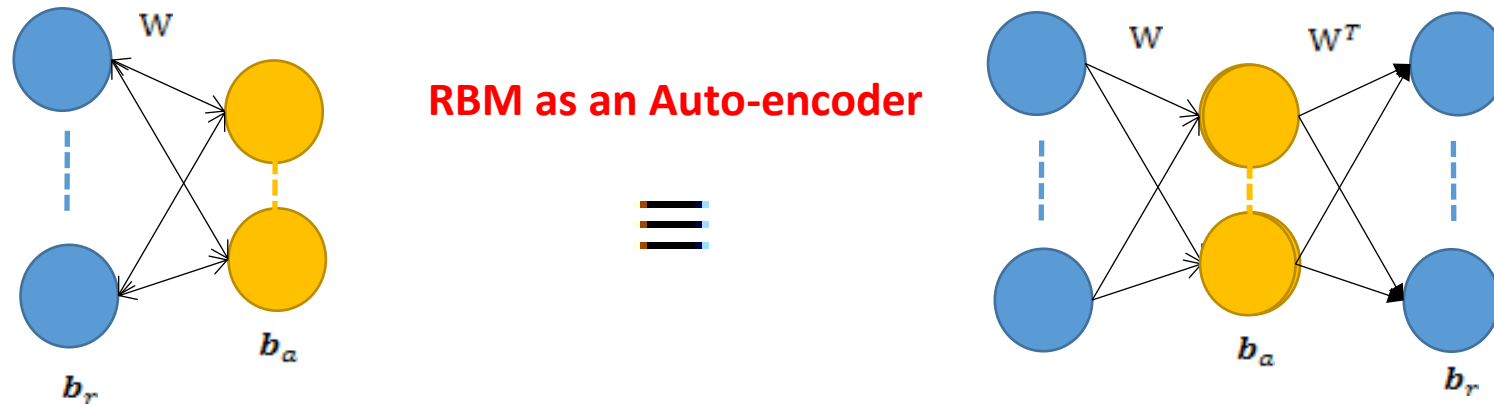
- An RBM is another example of neural network utilized in **dimensionality reduction**.
- It includes only two layers: visible layer and hidden layer.
- It is aimed that by using only one layer of neurons, correlations are partially removed from the input space, “**x**”, and the new space “**a**” is appeared in lower dimension (encoding part).
- For decoding part, RBM is used in inverse direction from hidden layer to visible layer.



$$a = \sigma(Wx + b_a)$$

RBM versus Auto-Encoders

- Unlike Auto-Encoder the decoding part is not independent from encoding part.
- In RBM decoding part is the same encoding part in which inputs and outputs are replaced with each others.



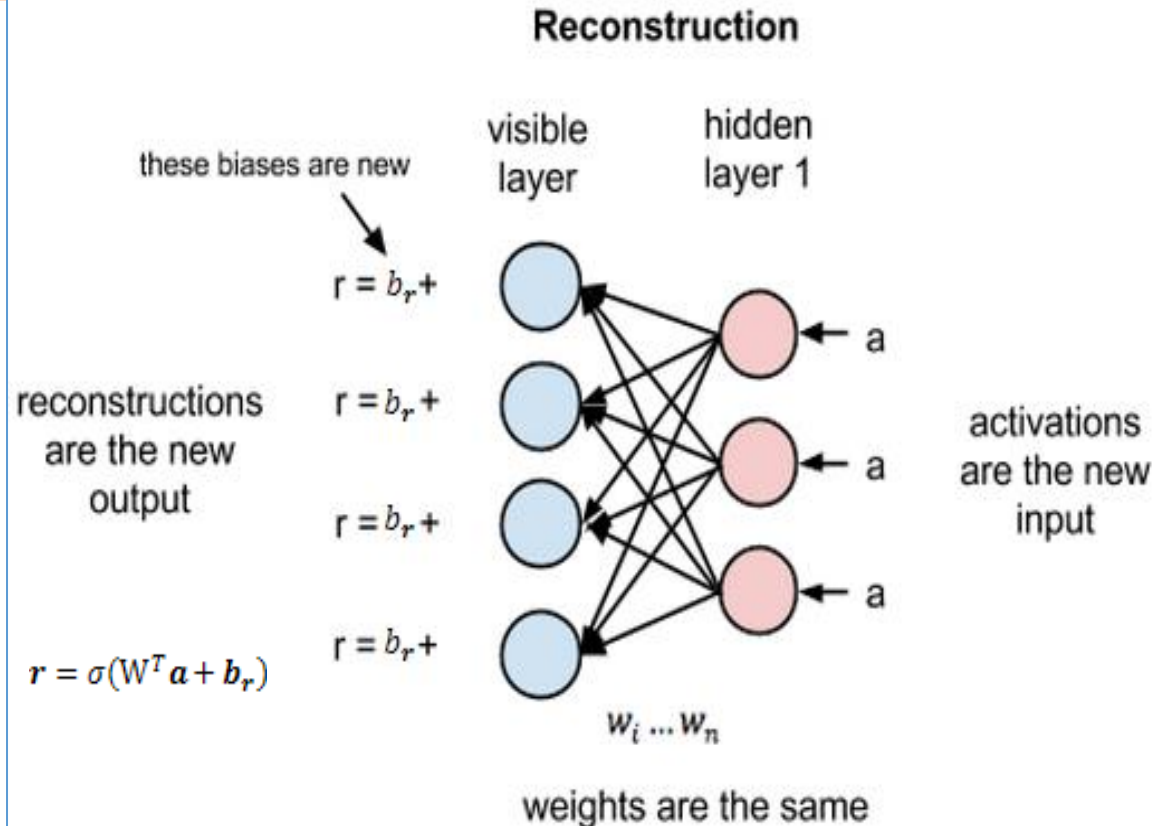
Training of an RBM

- RBM similar to an Auto-Encoder is learned by trying to retrieve (reconstruct) the initially applied pattern (decoding part).

Optimization Problem:

$$W = ? \quad b_a = ? \quad b_r = ? : r \rightarrow a$$

$$x \leftrightarrow \sigma(W^T \sigma(Wx + b_a) + b_r)$$



1. Learning by Gradient Search

$$J(W, \mathbf{b}_a, \mathbf{b}_r) = 0.5 \sum_{i=1}^m \|\mathbf{x}^i - \sigma(W^T \sigma(W \mathbf{x}^i + \mathbf{b}_a) + \mathbf{b}_r)\|^2$$

$$W^+ = W^- - \alpha \nabla J_W(W, \mathbf{b}_a, \mathbf{b}_r)$$

$$\mathbf{b}_a^+ = \mathbf{b}_a^- - \alpha \nabla J_{\mathbf{b}_a}(W, \mathbf{b}_a, \mathbf{b}_r) \quad \mathbf{b}_r^+ = \mathbf{b}_r^- - \alpha \nabla J_{\mathbf{b}_r}(W, \mathbf{b}_a, \mathbf{b}_r)$$

2. Learning by Gibbs Sampling

$$(1) \quad x \rightarrow a \quad \Delta W = \varepsilon(\mathbf{a} \mathbf{x}^T - \hat{\mathbf{a}} \hat{\mathbf{x}}^T)$$

$$(2) \quad \hat{x} \leftarrow a \quad \Delta \mathbf{b}_r = \varepsilon(\mathbf{x} - \hat{\mathbf{x}})$$

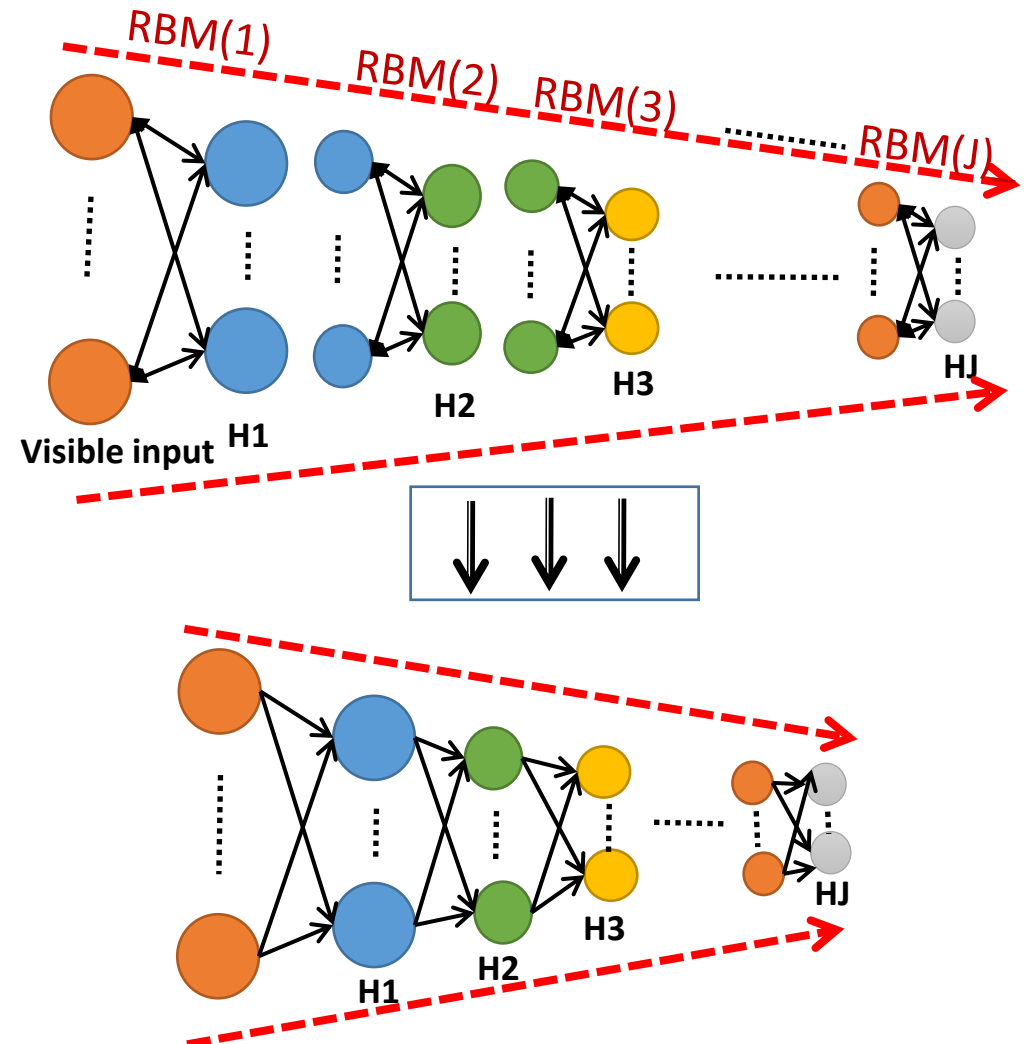
$$(3) \quad \hat{x} \rightarrow \hat{a} \quad \Delta \mathbf{b}_a = \varepsilon(\mathbf{a} - \hat{\mathbf{a}})$$

1. since derivation of the “sigmoid” or “tanh” is a positive scalar (less than one), the sign of gradient is affected directly by the output error and weights.
2. The resulted positive scalar from the derivation can be ignored.

Cascaded RBMs

Method

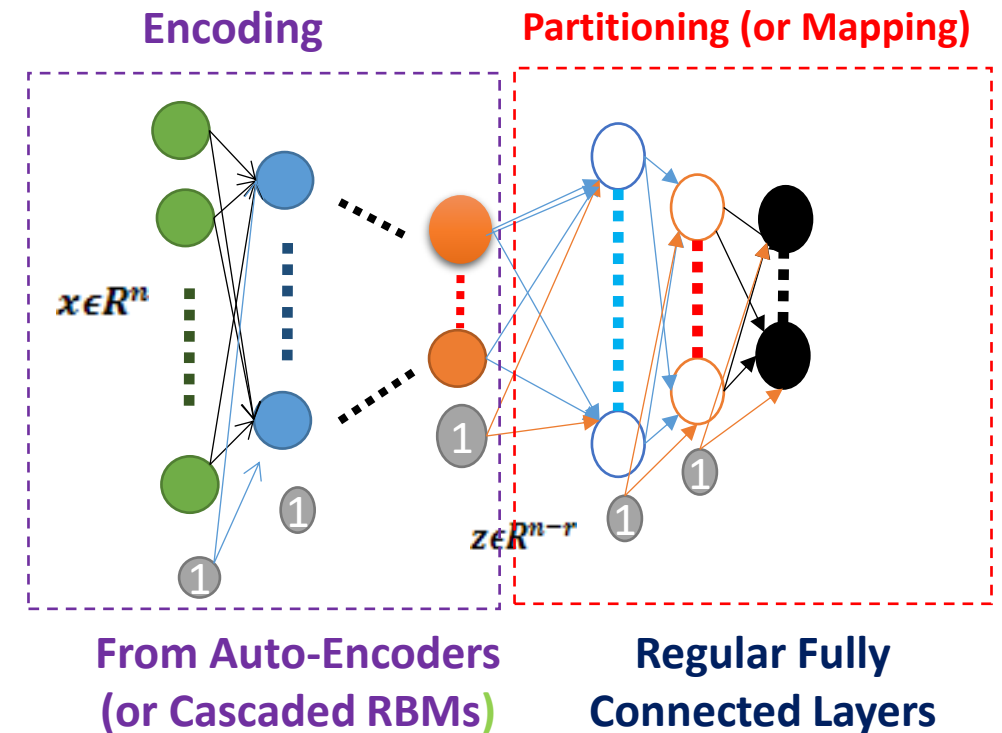
1. Considering the given inputs, train the first RBM.
2. Consider the Hidden layer of the last trained RBM as new inputs.
3. Train a new RBM.
4. If the dimension of the hidden layer is less than the number of inputs, consider the trained RBM as an approved RBM and then return to step 2, else go to next step.
5. Stack RBMs from the first trained one until the last approved one.
6. End.



Using Auto-Encoders (or Cascaded RBMs) in Classification and Regression

Deep Belief Network (DBN)

1. Define and train (1) an Auto-Encoder or (2) a Cascaded RBM for dimensionality reduction of the input space
2. Consider the encoding part of the Auto-encoder (or integrated RBMs) as initial layers of a MLNN.
3. Add one or two more hidden fully connected layers to provide the required partitioning or mapping (respectively utilized in Classification or Regression problems)
4. Such NNs are a variant of Deep Learning Networks.
5. Hinton in 2009 introduced and named such networks as Deep Belief Network (DBN) .

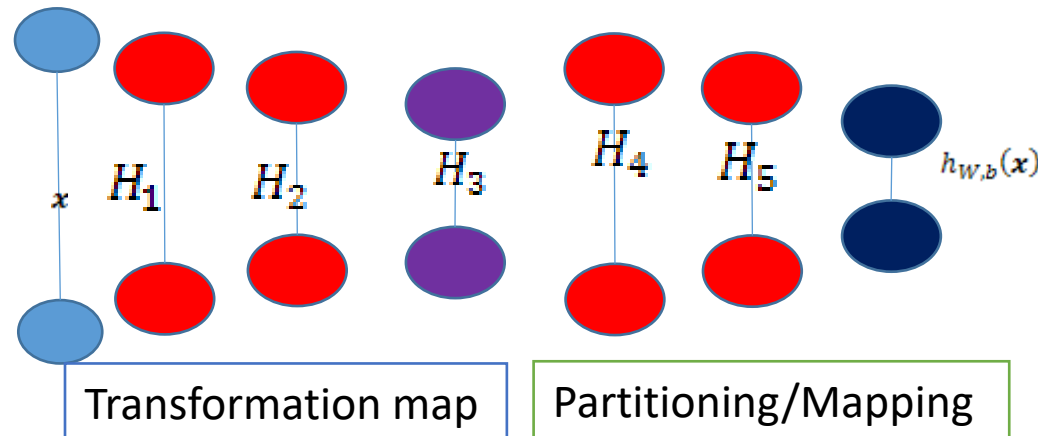


Some Important Notes about Deep Belief Network (DBN)

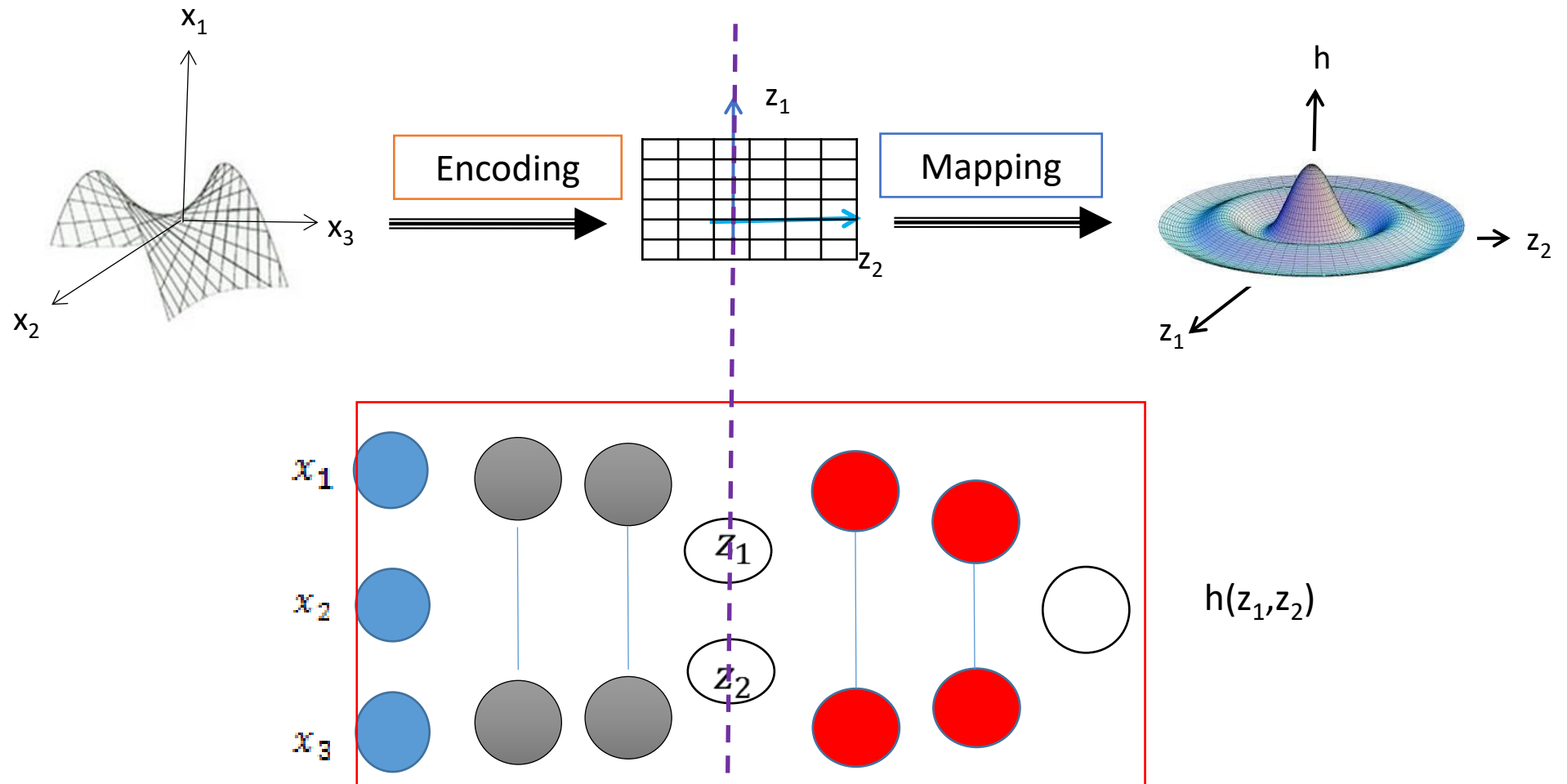
1. A **DBN** includes two parts: Encoding part and partitioning or mapping part.
2. The encoding part is trained through an auto-encoder(or cascaded RBMs) without using the target outputs (unsupervised).
3. However, The second part is trained by target outputs (supervised).
4. One can tune all parameters of the **DBN** by error back propagation (Fine Tuning).

How many hidden layers are required in DBN (at least)?

- For Encoding part, since it is aimed to learn the transformation function for Dimensionality Reduction, (in the worse case) three layers including two hidden layers and the output layer are required.
- It was discussed that for partitioning or mapping cases (in the worse case), two hidden layers are enough.
- Hence, a MLNN with 5 hidden layers is good enough for Dimensionality Reduction (at first part) and partitioning or mapping problems (at second part).



An illustrative Example



End of Chapter2

Thank you